



TECHNICAL UNIVERSITY OF CRETE
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
FIELD OF SOFTWARE ENGINEERING

Federated Learning With Geometric Approach at Flower Framework.

Implementation and Simulation

DIPLOMA THESIS

of

PANAGIOTIS G. SKLAVOS



Supervisor: Antonios Deligiannakis
Professor

Committee: Vasilios Samoladas
Associate Professor

Committee: Nikolaos Giatrakos
Assistant Professor

Chania, April 2024



TECHNICAL UNIVERSITY OF CRETE
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
FIELD OF SOFTWARE ENGINEERING

Federated Learning With Geometric Approach at Flower Framework.

Implementation and Simulation

DIPLOMA THESIS

of

PANAGIOTIS G. SKLAVOS

Approved by the examination committee on 28th April 2024.

(Signature)

(Signature)

(Signature)

.....
A. Deligiannakis
Professor

.....
V. Samoladas
Associate Professor

.....
N. Giatrakos
Assistant Professor

Chania, April 2024



TECHNICAL UNIVERSITY OF CRETE
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
FIELD OF SOFTWARE ENGINEERING

Copyright © – All rights reserved.
Panagiotis G. Sklavos, 2024.

The copying, storage and distribution of this diploma thesis, exall or part of it, is prohibited for commercial purposes. Reprinting, storage and distribution for non - profit, educational or of a research nature is allowed, provided that the source is indicated and that this message is retained.

The content of this thesis does not necessarily reflect the views of the Department, the Supervisor, or the committee that approved it.

DISCLAIMER ON ACADEMIC ETHICS AND INTELLECTUAL PROPERTY RIGHTS

Being fully aware of the implications of copyright laws, I expressly state that this diploma thesis, as well as the electronic files and source codes developed or modified in the course of this thesis, are solely the product of my personal work and do not infringe any rights of intellectual property, personality and personal data of third parties, do not contain work / contributions of third parties for which the permission of the authors / beneficiaries is required and are not a product of partial or complete plagiarism, while the sources used are limited to the bibliographic references only and meet the rules of scientific citing. The points where I have used ideas, text, files and / or sources of other authors are clearly mentioned in the text with the appropriate citation and the relevant complete reference is included in the bibliographic references section. I fully, individually and personally undertake all legal and administrative consequences that may arise in the event that it is proven, in the course of time, that this thesis or part of it does not belong to me because it is a product of plagiarism.

(Signature)

.....
Panagiotis G. Sklavos

28th April 2024

Abstract

Here goes the abstaract of the thesis

Keywords

key,words

Preface

Preface content . . .

to my parents

Acknowledgements

Acknowledgements content . . .

Chania, April 2024

Panagiotis G. Sklavos

Table of Contents

Abstract	vi
Preface	vii
Acknowledgements	ix
List of Abbreviations	xv
1 Introduction	1
1.1 Background & Motivation	1
1.2 Problem Statement	1
1.3 Objectives & Methodology	1
1.4 Thesis Outline	1
I Literature Review	2
2 Theoretical Background	3
2.1 Fundamentals of Machine Learning	3
2.1.1 Bridging Machine Learning and Artificial Intelligence	3
2.1.2 The Taxonomy Machine Learning Techniques	3
2.1.3 Model Complexity & Challenges	6
2.2 Artificial Neural Networks	7
2.2.1 Perceptron and Activation Functions	7
2.2.2 Network Architectures	10
2.2.3 Learning in Neural Networks	13
2.3 Optimization Techniques	14
2.3.1 Gradient Descent And Its Variants	14
2.3.2 Towards ADAM Optimizer	16
2.4 Distributed Machine Learning	18
2.4.1 The Need for Distribution	18
2.4.2 Distributed Learning Systems Architecture	19
2.4.3 Challenges in Distributed Machine Learning	22
2.5 Federated Learning	23
2.5.1 Principles of Federated Learning	24
2.5.2 Federated Learning Categorizations	25
2.5.3 Federated Learning Methodology	26
2.5.4 Problem Formulation in Federated Learning	27
2.6 Networking in Federated Systems	30
2.6.1 Network Topologies	30
2.6.2 Communication in Federated Systems	30
2.6.3 Communication Performance Metrics	31

2.6.4	Simulating Real-World Networks	33
2.7	Summary & Transition to Related Work	34
3	Related Work	35
3.1	Datasets in Federated Learning Research	35
3.1.1	Benchmark Datasets	35
3.1.2	Real-world Datasets	36
3.1.3	IID VS. non-IID Data Distributions	37
3.2	ANN Architectures	38
3.2.1	LeNet-5	38
3.2.2	VGG16	38
3.2.3	ResNet	39
3.3	Federated Learning Algorithms	40
3.3.1	Federated Stochastic Gradient Descent (FedSGD)	40
3.3.2	Federated Averaging (FedAvg)	41
3.3.3	Contemporary Challenges of Federated Learning	42
3.3.4	Countermeasures Introduced in Thesis	43
3.4	Synchronization Techniques	44
3.4.1	Geometric Method	44
3.4.2	Functional Dynamic Averaging	48
3.4.3	Linear Approximation	52
3.4.4	Sketch FDA	53
3.4.5	Multiprogramming & Synchronization Primitives in DML	54
3.5	NS3-FL: Bridging Data & Network Management	55
3.5.1	Framework Overview	56
3.5.2	Learning, Network, and Power Models	56
3.5.3	Implementation	57
3.5.4	Simulation Results	58
3.5.5	Conclusion	58
II	Technical Implementation	59
4	Environment Replication	60
4.1	Tensorflow & Keras Deep Learning Platforms	60
4.1.1	TensorFlow Framework	60
4.1.2	Keras Library	61
4.2	Flower Framework: An Efficient Approach to FL	61
4.3	NS3: An Event-Driven Network Simulator	63
4.4	Posix Sockets & gRPC	64
4.4.1	Posix Sockets	64
4.4.2	gRPC	64
5	Methodology	65
5.1	System Architecture	65
5.1.1	Component Overview	65
5.1.2	Interaction Flow	65
5.2	Dataset Preprocessing & Management	65
5.2.1	Dataset Sources and Selection Criteria	65
5.2.2	Preprocessing Techniques	65

TABLE OF CONTENTS

5.3	Model Development and Training	65
5.3.1	Model Architecture	65
5.3.2	Training and Optimization	65
5.4	Federated Learning Implementation	65
5.4.1	Client-Server Setup	66
5.4.2	Data Partitioning and Distribution	66
5.4.3	Model Aggregation	66
5.5	Communication Protocols and Data Serialization	66
5.5.1	Implementation of Communication Protocols	66
5.5.2	Data Serialization Technique	66
5.6	Network Simulation Integration	66
5.6.1	Simulation Tool Configuration	66
5.6.2	Simulating Network Conditions	66
5.6.3	Integration with Federated Learning	66
5.7	Evaluation Metrics and Performance Analysis	66
5.7.1	Model Evaluation Metrics	66
5.7.2	Network Simulation Metrics	66
5.7.3	Performance Analysis	66
6	Experimental Results & Discussion	67
6.1	Overview of Experimental Results	67
6.2	Analysis of FL performance	67
6.2.1	Accuracy and Efficiency	67
6.2.2	Scalability and Robustness	67
6.3	Network Simulation Findings	67
6.4	Discussion on Designed System Efficacy	67
6.4.1	Contributions to Federated Learning Research	67
6.4.2	Implications for Real-World Applications	67
6.4.3	Comparison with Technologies	68
6.5	Limitations and Challenges	68
III	Epilogue	69
7	Conclusions	70
7.1	Research Conclusions	70
7.2	Contributions of This Thesis	70
7.3	Recommendations for Future Research	70
7.4	Final Thoughts	70
	Bibliography	75

List of Figures

2.1	Multi-dimensional taxonomy of Machine Learning[1].	4
2.2	Simple representation of Rosenblatt's perceptron.	8
2.3	A simple MLP example with n input neurons, one hidden layer with $h_1 = 3$ neurons and $m = 1$ output neuron.	9
2.4	A side by side illustration of FFNN and RNN architectures.	11
2.5	A CNN sequence to classify handwritten digits[2]	11
2.6	A convolution of a 4×4 image with a 3×3 kernel highlighting the sliding window operation[3]	12
2.7	Demonstration of max and average pooling operations.	12
2.8	Gradient Descent Algorithm[4]	14
2.9	The effects of unsuccessful learning rate selection[5]	16
2.10	Trajectories in ravine surfaces, with and without momentum[6].	16
2.11	A comparison between distributed and centralized learning approaches	18
2.12	Data and Model Parallelism Methods	19
2.13	Principle Distributed Systems Communication Topologies	20
2.14	Federated Learning Characteristics.	24
2.15	Federated Systems separation in Cross-Device or Cross-Silo scale	25
2.16	Federated Systems separation in horizontal or vertical learning[7].	26
2.17	General FL training process[8].	28
3.1	Features of a good dataset for machine learning	35
3.2	Illustration of Fashion-MNIST samples	36
3.3	Example of IID and non-IID distributions of MNIST: In the IID dataset, each device contains a similar and random sample of digits (0 – 9), ensuring that the data distribution on each device is similar to the overall distribution. In the Non-IID dataset, each device contains digits that are not uniformly distributed; instead, each device has a concentration of certain digits. For example, one device has mostly 0s, another mostly 7s, and so on	37
3.4	LeNet's 5 Architecture.	38
3.5	VGG16 Architecture.	39
3.6	ResNet's Architecture with residual learning block illustration.	39
3.7	Drift vectors comprising the convex hull.[9]	47
3.8	Architectural overview of ns3-fl.	56
3.9	Synchronous federated learning training workflow.	57
4.1	Flower architecture with a server, two edge clients and one virtual client.	62

List of Tables

2.1	Learning types based on feedback along with key examples of each field	5
2.2	Commonly used activation functions in neural networks[10]	10
2.3	Challenges of Distributed Machine Learning	23
3.1	Notation used in Federated Learning algorithms	40
3.2	Summary of socket commands used in ns3-fl.	57
3.3	Comparison of computational time (in seconds) between real deployment and ns3-fl simulations on different datasets.	58

List of Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
ANN	Artificial Neural Networks
MLP	Multi-Layer Perceptron
FFNN	Feed-Forward Neural Networks
CNN	Convolutional Neural Networks
RNN	Recurrent Neural Networks
MSE	Mean Squared Error
GD	Gradient Descent
BGD	Batch Gradient Descent
SGD	Stochastic Gradient Descent
MBGD	Mini-Batch Gradient Descent
ADAM	Adaptive Moment Estimation
DML	Distributed Machine Learning
GPU	Graphics Processing Units
TPU	Tensor Processing Units
IID	Independent and Identically Distributed
FL	Federated Learning
HFL	Horizontal Federated Learning
VFL	Vertical Federated Learning
FedAvg	Federated Averaging
GM	Geometric Monitoring
FGM	Functional Geometric Monitoring
FDA	Functional Dynamic Averaging
RTC	Round Terminated Condition
API	Application Programmable Interface

Chapter **1**

Introduction

1.1 Background & Motivation

1.2 Problem Statement

Talk about Networking overhead and effect on learning performance

1.3 Objectives & Methodology

Present the FDA approach and geometric technique and how will be utilized to restrict the need of Networking communication

1.4 Thesis Outline

Part I

Literature Review

Chapter 2

Theoretical Background

The following chapter offers a comprehensive overview of the key theoretical concepts integral to this research study, as previously outlined. It systematically presents the scientific principles amassed, to cultivate a unified understanding of prior knowledge that made this research feasible. The discussion begins with broader terminologies and progressively delves into more complex notions that will be explored in subsequent sections. This structured approach aims to facilitate a deeper understanding of the theoretical underpinnings essential to this research while highlighting the underlying connection among the various terms presented.

2.1 Fundamentals of Machine Learning

2.1.1 Bridging Machine Learning and Artificial Intelligence

In the rapidly evolving domain of computer science, Artificial Intelligence (AI) and Machine Learning (ML) exhibit a compelling interrelationship, particularly promising in an era abundant with data. As the demand for intelligent applications escalates, it becomes imperative to distinguish clearly between these two frequently conflated terms. As critically highlighted by Kühn et al.[11], the terms AI and ML are often used interchangeably both in scholarly literature and expert discussions, leading to considerable ambiguity.

Essentially, ML can be seen as a subset of AI, primarily focused on designing systems that autonomously learn from data and enhance their performance over time without explicit reprogramming. In contrast, a widely acknowledged definition of AI, as provided by Russell and Norvig[12], positions AI as "the study of agents that receive percepts from the environment and perform actions." On this basis, the authors classify AI systems into four types: Those that think like humans, think rationally, act like humans, and act rationally.

Building on this classification, it provides a fundamental framework for understanding the diverse objectives and methodologies within AI research. It emphasizes AI's ability to perceive its environment through data acquisition and respond through actions, thereby linking it closely with fields such as machine learning, robotics, natural language processing, and reasoning. This clear delineation not only helps in understanding AI's broad applications but also illustrates the foundational role of ML within the broader AI landscape.

2.1.2 The Taxonomy Machine Learning Techniques

Now that a clear distinction between the terms AI and ML is established, it is crucial to examine the nature of the diverse methodologies encompassed by ML. The generalisability of published research on this issue remains a challenge. Over recent years, various studies have offered differing classifications of ML methods, with the determining factor being the scope under

which ML was investigated. Predominantly, existing literature has been concentrated on the nature of the feedback mechanism applied, primarily categorizing ML algorithms into supervised, unsupervised and semi-supervised [13, 14] with some studies also incorporating reinforcement learning into their analysis [15, 16].

This research, however, adopts a more holistic approach, aspiring to grasp the whole spectrum of ML applications. Instead of focusing solely on the nature of the learning feedback, this classification also considers the objectives of the learning tasks and the timing of data availability. This broader taxonomy, initially introduced by Zhou et al.[1], encapsulates all the critical elements that influence the design and deployment of ML algorithms. The suggested taxonomy is the one shown below in fig. 2.1.

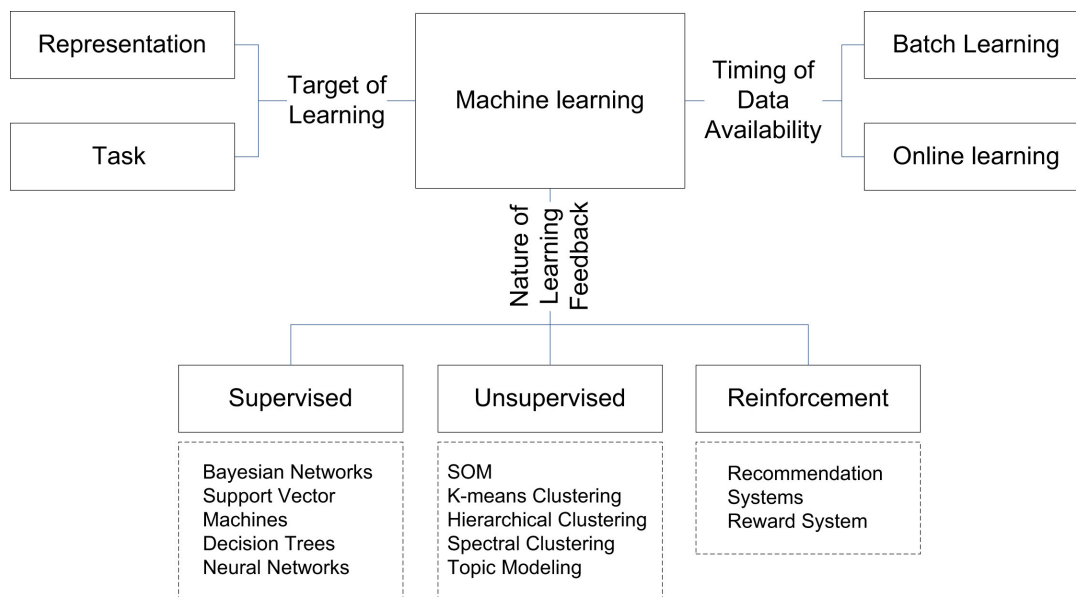


Figure 2.1. Multi-dimensional taxonomy of Machine Learning[1].

♦ **Nature of Learning Feedback:** Based on this dimension of the taxonomy applied to Machine Learning, three principal system categories and one secondary system emerge: supervised, unsupervised, and reinforcement learning systems and semi-supervised (secondary). In supervised learning, sample input-output pairs are presented to the system, which in turn attempts to infer a function able to map inputs to corresponding outputs. In practical terms, supervised ML techniques strive to construct a predictive model by applying algorithms to a set of known data points to extrapolate insights to an unseen dataset[16, 17].

Unsupervised learning, in contrast, can be identified as a more data-driven approach, where datasets devoid of labels are analyzed. This means that while the structure of the data is apparent; no explicit guidance is provided as to the expected outcome and the aim is to uncover inherent patterns within the data without predefined answers.

Likewise, reinforcement learning is not provided with input-output pairs, thus resembling unsupervised learning. Yet it requires a form of feedback—namely rewards or penalties linked to the actions taken. This environment-driven feedback mechanism guides the learning process towards optimal decision-making in an effort to minimize loss or maximize reward, depending on the environment’s setup[18]. The distinct applications of these techniques are demonstrated by Qiu et al.[15], who assert that supervised and unsupervised learning primarily concentrates on data analysis, whereas reinforcement learning is better suited for decision-making tasks.

Additionally, semi-supervised learning which sits at the intersection of supervised and unsupervised learning, entails learning from a limited set of labeled data complemented by a larger pool of unlabeled data. The aim of this learning method is similar to supervised learning, yet the knowledge is gathered both from annotated and unannotated data. Some of the key algorithmic examples of each category are demonstrated in the following table 2.1;

Learning type	Model building	Examples
Supervised	Algorithms learn from labeled data (task-driven)	Classification, regression
Unsupervised	Algorithms learn from unlabeled data (data-driven)	Clustering, associations
Semi-supervised	Models built using combined data (labeled + unlabeled)	Classification, clustering
Reinforcement	Models based on reward or penalty (environment-driven)	Classification, control

Table 2.1. *Learning types based on feedback along with key examples of each field*

♦ **Target of Learning:** Machine learning can be segmented into representational and task learning based on the target of learning—whether it focuses on input features or specific tasks respectively. Representational learning seeks to identify novel data representations that simplify the extraction of useful information for constructing classifiers or other predictors[19]. This approach frequently entails isolating critical features that display firm influence upon data variability, which can be proven exceptionally practical in probabilistic models aiming to capture the posterior distributions of fundamental factors. Common processes in this category include density estimation, which attempts to define the underlying probability density of data and dimensionality reduction, aimed at reducing the complexity of data from a higher to a lower-dimensional space. Last but not least, another fundamental example of representational learning is deep learning which is one of the fastest-growing fields of ML providing a viable approach for big data processing. All things considered though, defining clear objectives in representational learning can be challenging.

On the contrary, task learning is more outcome-oriented, targeting definite outputs, and can be recognized both in supervised and un-supervised setups. To offer an illustration, classification and regression, generally categorized under supervised learning, aim to predict discrete classes and continuous values respectively[20], whereas clustering—a form of unsupervised learning—groups data without predefined categories[21]. All things considered, each technique addresses specific analytical needs and applications within the broader ML framework.

♦ **Timing of Data Availability:** On the basis of data availability timing, Machine Learning can be classified into online and batch learning. Briefly, batch learning algorithms consolidate knowledge by training comprehensively on the entirety of the dataset available at a certain point in time, whereas online learning algorithms process data continuously in streams, accommodating updates through individual samples or mini-batches[22].

To elucidate these methodologies, in the offline learning scenario typical of batch processing, once training is complete and the model is exhibiting adequate performance on the test set, the model is finalized and deployed. In case of new data becoming available, indicating a need for model recalibration, a new cycle of training and evaluation needs to be performed integrating both existing and new data. Another key factor to note is that the batch learning process functions under the belief that data are Independent and Identically Distributed (IID) or that

data are drawn from the same probability distribution[23]. Nonetheless, this assumption often diverges from the complexities of real-world data dynamics.

Conversely, online learning operates under no statistical assumptions and adapts continuously to data, rendering it indispensable in environments where data generation is ceaseless or complete dataset training is impractical. Last but not least, another principal difference between the two ML categorizations is that batch training is expected to generalize, producing a more broad reflection of knowledge, while this principle does not uniformly apply to online learning, which focuses on immediate, accurate predictions on incoming data data[23].

All things considered, the above multi-dimensional taxonomy can be applied to any given machine learning algorithm, resulting in a categorization along the three discrete axes displayed at fig. 2.1. For instance, conventional decision trees belong to supervised, task-oriented, batch-learning algorithms.

2.1.3 Model Complexity & Challenges

In an effort to measure the efficiency and generalizability of Machine Learning algorithms, two crucial terms to consider are model performance and complexity. Model performance refers to the effectiveness of a model in making accurate predictions or decisions based on new, unseen data[24]. Performance is typically assessed using metrics such as accuracy for classification or mean squared error for regression. For the purposes of this study, accuracy is the observed metric and can be calculated as follows:

$$accuracy = \frac{CorrectObservations}{TotalObservations} \quad (2.1)$$

In addition to quantifying efficiency, model performance provides a tangible estimator for whether a model is improving or not with new data while enabling different ML algorithm comparisons under a common scope.

Model Complexity

Model complexity relates to a model's ability to represent a wide range of functions, primarily influenced by its architecture, such as the number of parameters, the depth, and the types of functions it can incorporate. Essentially, complexity determines the model's capacity to capture intricate patterns within the data. Increasing a model's complexity means introducing more parameters, thereby enhancing its adaptability to diverse patterns. However, this increase comes at a cost: while the risk of underfitting diminishes, the danger of overfitting becomes significant, as the model might become excessively tailored to the training dataset, making it too complex for the data it needs to generalize from[25, 26].

Balancing Challenges: Overfitting and Underfitting

The key to optimizing model performance lies in carefully adjusting its complexity to find a balance between overfitting and underfitting. Overfitting happens when a model captures not only the underlying patterns but also the noise within the training data, leading to excellent performance on the training set but poor generalization to new, unseen data [25, 27]. Conversely, underfitting occurs when the model is too simplistic, failing to capture the true patterns in the data, which results in suboptimal performance on both the training and test datasets. This issue often arises when the model lacks sufficient complexity to learn the relationships between input features and the target variable.

Techniques to Achieve Generalizability

To find the optimal complexity where the model effectively extracts valuable patterns from the data without overfitting, several techniques are commonly employed:

- ◆ *Regularization*: This involves adding a penalty term to the loss function to prevent the model from becoming overly complex. Techniques like L1 (Lasso) and L2 (Ridge) regularization are widely used to achieve this.
- ◆ *Cross-validation*: This technique evaluates model performance on unseen data by partitioning the dataset into multiple training and validation sets, and averaging the evaluation metrics across all partitions to ensure robust performance.
- ◆ *Early stopping*: This involves monitoring the model's performance on a validation set during training and halting the training process when the performance on the validation set starts to decline, thereby preventing overfitting.

These strategies are essential for maintaining the delicate balance between overfitting and underfitting, ensuring that the model remains generalizable and performs well on new, unseen data[28].

2.2 Artificial Neural Networks

Artificial Neural Networks (ANNs) are a pivotal research area in Machine Learning, forming the backbone of deep learning, which has transformed handling vast and complex datasets. ANNs mimic the human brain's architecture, consisting of layers of interconnected neurons designed for specific computational tasks [29]. ANNs have significantly advanced fields such as image and speech recognition, natural language processing, and decision-making under uncertainty. Their versatility extends to various applications, including healthcare diagnostics, autonomous vehicles, and financial modeling, demonstrating robustness and adaptability[30]. This study focuses on online learning ANNs applying supervised learning for classification.

2.2.1 Perceptron and Activation Functions

Rosenblatt and Multi-Layer Perceptrons

Perceptrons, regarded as one of the earliest and simplest forms of artificial neurons, were initially introduced by Frank Rosenblatt in 1958 [31]. Perceptrons functionality is closely related to classification tasks and can be divided into two categories; Rosenblatt Perceptrons and Multi-Layer Perceptrons (MLP).

The first definition of perceptrons resembles a single-layer neural network with m input sources and one sole output neuron, where stimuli can be classified into two classes, using a linear decision boundary based on the Heaviside function. This single-layer structure, restricts Rosenblatt perceptron flexibility, limiting its functionality to linearly separable problems and binary classification tasks. To demonstrate the functionality of this kind of perceptron, a more strict mathematical description seems necessary[32]:

$$\begin{cases} v = \sum_{i=1}^m w_i x_i + b, \\ y = \phi(v) \end{cases} \quad (2.2)$$

Expanding on the definition provided by eq. (2.2), Consider x_i (for $i = 1, \dots, m$) as the set of input signals delivered to a perceptron. In this model, each w_i represents the synaptic weight associated with the perceptron for the corresponding input x_i . The term b is known as the

externally set bias. The term v is referred to as the *induced local field* or simply the linear combiner of inputs and weights. The function ϕ , which is an activation function or a step function, processes this induced local field to produce the output y (see fig. 2.2) of the perceptron. This perceptron model essentially consists of a linear combiner capped by a step function, which decisively determines the output based on the calculated local field v .

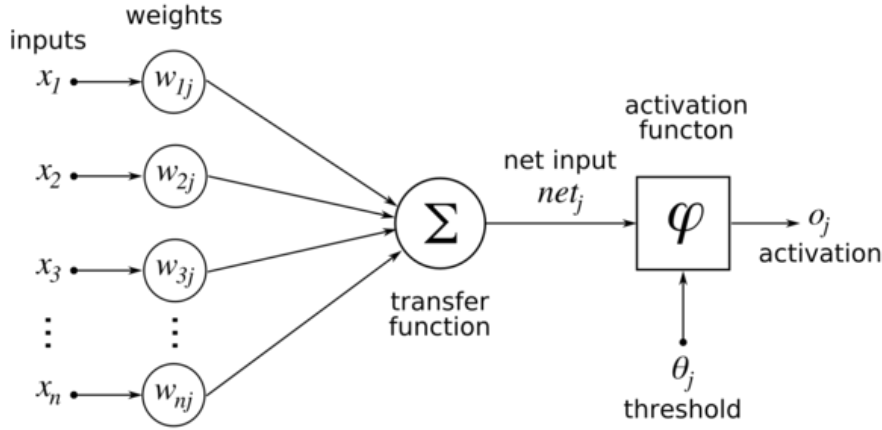


Figure 2.2. Simple representation of Rosenblatt's perceptron.

The activation function of Rosenblatt's perceptron, also referred to as the Heaviside step function, is a straightforward threshold function summarized by eq. (2.3). The binary output of the function allows us to determine whether an input sample belongs to one out of two classes limiting strictly in binary classification problems [33]. For this reason, more adaptable activation functions were introduced for various use cases which will be outlined later:

$$\phi(v) = \begin{cases} 1 & \text{if } v > 0, \\ 0 & \text{if } v \leq 0. \end{cases} \quad (2.3)$$

Despite the foundational impact of Rosenblatt's perceptrons in neural networks, their limitations with non-linear problems created a need for a more sophisticated approach. This led to the development of Multi-Layer Perceptron (MLP) models, which addressed the famous XOR problem highlighted by Marvin Minsky and Seymour Papert[34]. By incorporating multiple layers of neurons, including hidden ones, MLPs can learn intricate patterns in data, enhancing their learning capabilities and handling non-linear transformations. This advancement improved the network's generalization ability by learning hierarchical features and adapting to various levels of complexity, from linear problems to complex decision boundaries that single-layer perceptrons could not manage.

The architecture of a Multi-Layer Perceptron (MLP) differs substantially from Rosenblatt's perceptrons. It consists of three types of layers:

- ◆ *Input layer:* n source neurons.
- ◆ *Hidden layers:* one or several hidden layers each containing containing h_i neurons.
- ◆ *Output layer:* m output neurons.

Each neuron in the network employs a differentiable non-linear activation function. The most commonly utilized activation function in MLPs is the sigmoid which is close to linear near the origin while saturating rather quickly when getting away[33]. This allows multilayer perceptrons to model well both strongly and mildly nonlinear relations. Additionally, the structure of this

MLP is fully connected, meaning every neuron in a given layer is connected to all neurons in the subsequent layer (see fig. 2.3). Signals in the network flow unidirectionally from the input layer to the output layer, moving sequentially through each layer from left to right.

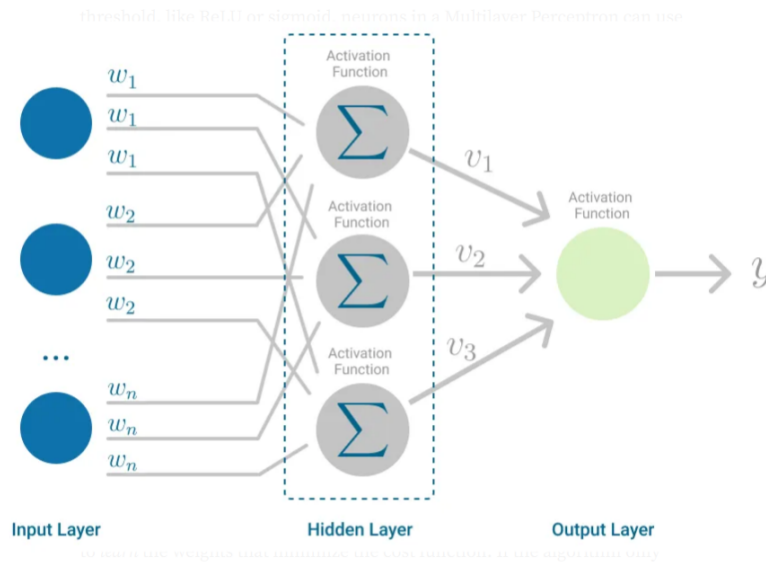


Figure 2.3. A simple MLP example with n input neurons, one hidden layer with $h_1 = 3$ neurons and $m = 1$ output neuron.

Evolution of Activation Functions

The functionality of neural networks is greatly influenced by the choice of activation functions. Initially, Rosenblatt’s perceptron used the Heaviside step function, a basic activation function that activates based on a simple threshold rule.

As neural network theory developed, more flexible activation functions were needed to enable learning in multi-layer architectures and address non-linear classification problems that single-layer perceptrons could not solve. The sigmoid function emerged as one of the most popular choices for Multi-Layer Perceptrons (MLPs) because of its continuous and differentiable nature, which is crucial for gradient-based learning algorithms like backpropagation.

Over the years, several other activation functions have been introduced to meet specific requirements of neural network architectures and problems. Common examples include the hyperbolic tangent (tanh), which is similar to the sigmoid but ranges from -1 to 1, centering the data; the Rectified Linear Unit (ReLU), popular in deep learning networks due to its efficiency and effectiveness at preventing the vanishing gradient problem[35]; and Softmax, predominantly used in the output layer of Convolutional Neural Networks (CNNs) to convert logits into a probability distribution over multiple exclusive classes (see section 2.2.2).

These activation functions have been extensively researched to maximize neural network performance[36, 10]. Table 2.2¹ presents a summary of commonly used activation functions.

These developments have significantly shaped the design and implementation of modern neural networks. The shift from simple threshold functions to complex non-linear functions has enabled neural networks to model intricate patterns in data, improving their ability to generalize from training data to unseen situations. This transition marks a pivotal evolution from early perceptrons to the sophisticated networks at the forefront of artificial intelligence research and applications today.

¹The indexed v_i in Softmax’s equation in the table symbolizes the reference to class i .

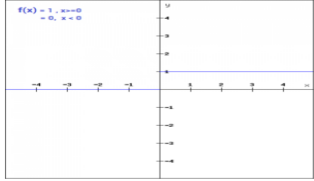
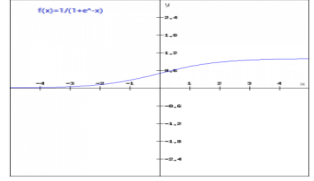
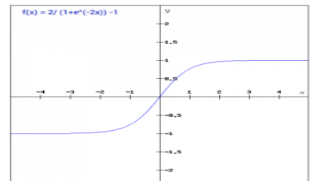
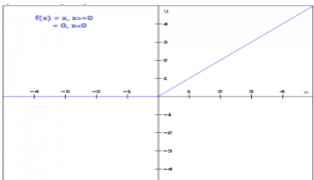
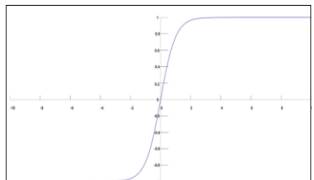
Function	Equation	Use	Graph
Heaviside Step	$\phi(v) = \begin{cases} 1 & \text{if } v > 0 \\ 0 & \text{if } v \leq 0 \end{cases}$	Binary Classification	
Sigmoid	$\phi(v) = \frac{1}{1+e^{-v}}$	Probabilities	
Hyperbolic Tangent (tanh)	$\phi(v) = \frac{e^v - e^{-v}}{e^v + e^{-v}}$	Centered Data	
Rectified Linear Unit (ReLU)	$\phi(v) = \begin{cases} v & \text{if } v > 0 \\ 0 & \text{if } v \leq 0 \end{cases}$	Sparse Activation	
Softmax	$\phi(v_i) = \frac{e^{v_i}}{\sum_k e^{v_k}}$	Multiclass Classification	

Table 2.2. Commonly used activation functions in neural networks[10]

2.2.2 Network Architectures

The research in the domain of neural networks always strives for optimization, and to be precise, maximization of performance and generalization[37]. Hence, in an effort to expand the limits of what can be achieved with Neural Networks, several diverse architectures have been proposed, tailored for specific data and fields. That way performance is improved by addressing the specific needs of a given problem. While the types of architectures developed to this day are countless, this analysis will be restricted to a brief overview of two major categories and afterward, there will be a shift in focus toward the architecture most valuable for this research.

First and foremost, there are Feed-Forward Neural Networks (FFNN)[40] which represents the simplest form of neural network, designed on the tracks of MLPs' logic discussed previously (see fig. 2.4a). In this type of networks, neurons are organized in layers with each neuron being connected to all neurons of subsequent layers. The network's flow is unidirectional from the input layer through hidden layers to the output layer. Common use examples contain image and speech recognition, due to their simplicity and effectiveness.

Moving forward, another crucial type of neural network architecture is Recurrent Neural Net-

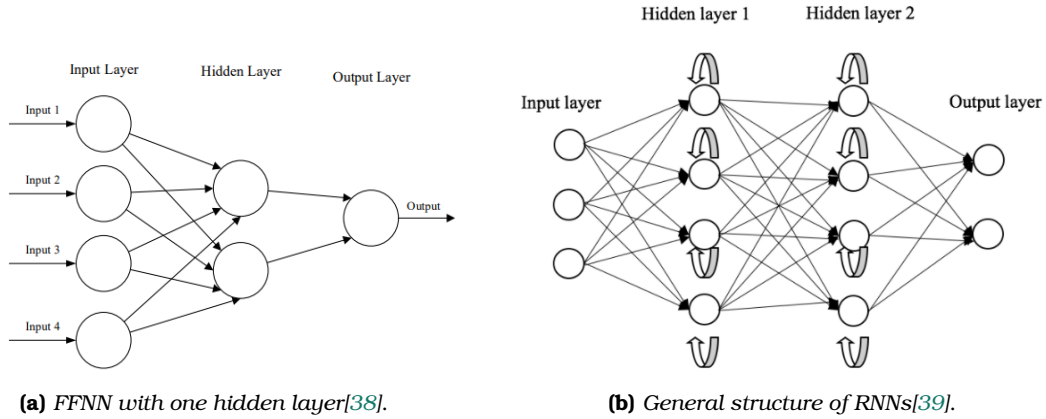


Figure 2.4. A side by side illustration of FFNN and RNN architectures.

works (RNNs)[39]. Recurrent Neural Networks are designed to handle sequential data[41]. Differing from traditional neural networks, RNNs process inputs using a loop mechanism that allows information from previous steps to influence current and future operations (see fig. 2.4b). This architecture [38] makes RNNs ideal for applications such as language modeling and text generation, where the sequence and context of information are critical. They are particularly effective in tasks like speech recognition, music generation, and other forms of sequential pattern recognition.

Convolutional Neural Networks (CNNs)

Building on the foundational principles of Feed-Forward Neural Networks, the exploration into network architectures takes a sophisticated turn with Convolutional Neural Networks (CNNs)[42], a significant achievement in Deep Learning. CNNs have broad applicability in fields such as image and video recognition, recommendation systems, image classification, image segmentation, natural language processing, and computer vision. The name of CNNs stems from the *convolution*-an mathematical operation described in calculus that combines two functions to produce a third, that expresses how the shape of one is modified by the other. This inherent property of convolutional layers provides CNNs with translation-invariance, enabling them to detect and extract patterns and features from data regardless of position, orientation, scale, or translation.

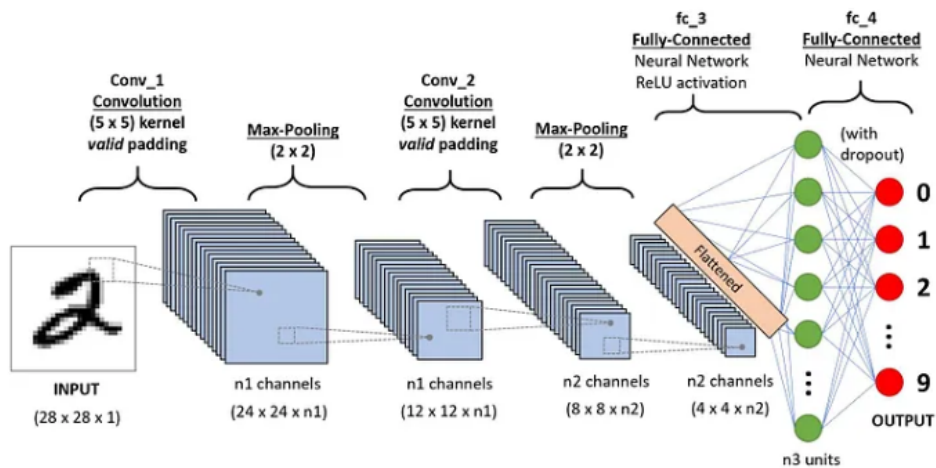


Figure 2.5. A CNN sequence to classify handwritten digits[2]

To further elucidate the methodology of a CNN, we will observe the example of fig. 2.5 depicting a single input from the MNIST dataset² while being processed. The format in which the system perceives the input is a pixel representation of the image. After the input is received the sample undergoes a series of critical operations that eventually lead to the system's output.

The *convolution layer* performs a sliding window function to the matrix of pixels representing the digit's image, commonly known as kernel or filter. Several filters of equal size are applied, and each filter is used to recognize a specific pattern from the image, generating multiple feature maps. Observing fig. 2.5, a $4(\text{height}) \times 4(\text{breadth})$ kernel is utilized, sliding over the input image, calculating the convolution result at each point and populating the feature map.

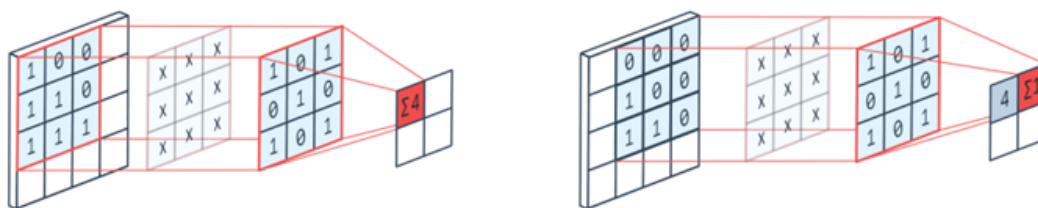


Figure 2.6. A convolution of a 4×4 image with a 3×3 kernel highlighting the sliding window operation[3]

After each convolution operation, a ReLU (see table 2.2) activation function is applied to deduce non-linear relationships between the features in the image, hence improving the network's capability for capturing complex patterns.

Following the convolution and activation steps, the *pooling (or subsampling) layer* reduces the spatial dimensions of the feature maps. This step is crucial as it decreases computational requirements and the number of parameters, helping prevent overfitting. Max pooling, which selects the maximum value from each patch of the feature map, is the most frequently used method (fig. 2.7a). Another common method is average pooling, which calculates the average value of each patch (fig. 2.7b). The final pooling layer converts the feature map into a flattened, one-dimensional array, preparing it for input into the fully connected layer.



Figure 2.7. Demonstration of max and average pooling operations.

The *fully connected layers* represent the last layer of a CNN and their inputs correspond to the flattened one-dimensional matrix generated by the last pooling layer. ReLU activation functions are applied to them for non-linearity again (as shown in fig. 2.5). Finally, a softmax prediction layer is used to generate probability values for each of the possible output labels, and the final label predicted is the one with the highest probability score.

²Dataset of handwritten digits

2.2.3 Learning in Neural Networks

Having established the principal components and architecture of ANNs, we proceed with a brief demonstration of their learning method. Learning in neural networks, achieved by training³, involves the adjustment of the network's internal parameters to minimize the discrepancy between the actual output and the target output. This process is crucial for developing models that can perform well on unseen data. Yet, In order to successfully grasp the reasoning behind the training process, it is crucial to decompose the procedure into smaller, comprehensive, sub-processes.[43]. In essence, this analysis partitions the training process into six parts as follows;

1. *Initialization*: The initialization of weights is a critical first step in the training of neural networks. Proper initialization sets the stage for an efficient and effective learning process, influencing how quickly a network converges to a solution, if at all. Initially, weights can be set randomly, but more sophisticated methods such as He initialization or Xavier initialization are commonly used to improve the convergence properties of deep networks[44]. These methods adjust the scale of the weights according to the number of input and output neurons, promoting an even distribution of activations across layers during the initial phases of training[45].
2. *Forward Propagation*: This step involves computing the output of a neural network, starting by feeding the input data and calculating the weighted sum of each neuron's inputs. The activation or non-activation of the neuron is determined by the output of the nonlinear activation function (see table 2.2) over the calculated sum. The process begins at the input layer and proceeds sequentially through the hidden layers and finally to the output layer, where a prediction is made. The specific transformations applied to the neurons' sums depend on the network's architecture (e.g., convolutional, recurrent) and the types of layers used[46].
3. *Loss Computation*: A cost (or loss) function, quantifies the error between the predicted outputs and the actual target values. The cost function selection can significantly affect the network's performance and convergence. Common choices include Mean Squared Error (MSE) for regression tasks and Cross-Entropy Loss for classification tasks. The cost function provides a performance measure used during backpropagation to adjust the model's weights.
4. *Back Propagation*: Backpropagation is the fundamental algorithm for training artificial neural networks. It calculates the gradients of the loss function with respect to the weights of the network by applying the chain rule of calculus. These gradients indicate the necessary adjustments to the weights to minimize the loss function and improve model accuracy. The process involves propagating the error backward through the network, updating the weights to reduce the prediction error for subsequent iterations[47].
5. *Gradient Descent*: The gradients calculated during the *back propagation phase* comprise a clear indication of the steepest ascent on the loss curve, meaning the direction of the maximizing loss. On the contrary, as the objective is to minimize the loss, it becomes obvious that the required update of the network's parameters needs to be performed moving in the opposite direction of the gradients. By iteratively adjusting the weights in this direction, the accuracy of the system increases, while The magnitude of this update is controlled by a learning rate hyperparameter of the optimization algorithm. Additional information for this matter will be presented in the following section 2.3.
6. *Iteration*: Steps 2 – 5 are repeated for each batch of training data for multiple epochs⁴ until the neural network's performance on the training data achieves convergence to a solution.

³Training is also called fitting in ML terminology

⁴Epoch is a complete pass over the training dataset

The above process is repeated iteratively until the breaking condition is satisfied. However, this does not necessarily guarantee generalization. The above statement loops back to a previously reported issue in machine-learning-overfitting (see section 2.1.3). Therefore, training can include a validation step where the model is tested on unseen data to adjust hyperparameters and prevent overfitting.

In closing analysis, it should be noted that the neural networks, as demonstrated, accumulate knowledge by iterative adjusting the network's parameters based on the optimization algorithm applied. This suggests that a vast number of operations need to be performed at each training step. In restricted-size models, the above operation may be a manageable load for the computers to handle, then again this is not the fact, for substantially sized networks, where the computational demands can become humongous. Hence, training neural networks is typically performed on powerful hardware or specialized hardware accelerators (GPUs or TPUs) to speed up the process[43].

2.3 Optimization Techniques

Optimization is, undoubtedly, a crucial matter in the field of Data Science and Machine Learning. This section explores various techniques stemming from optimization theory, aiming to enhance the performance of machine learning algorithms through the effective minimization of loss functions. Starting from the fundamental principles of Gradient Descent, and continuing by presenting the key variations relevant to this research approach, this analysis clarifies the role of optimizers in contemporary neural networks.

2.3.1 Gradient Descent And Its Variants

Principle of Gradient Descent

Gradient Descent and its Variants are classified as iterative optimization algorithms, based on the idea of iteratively adjusting the parameters (coefficients) of a model in the direction of the steepest descent with respect to the machine learning algorithm's cost function[48]. Essentially, the objective of the algorithm is to find the parameter values that minimize the cost function of the neural network. For this to be achieved, the gradient of the loss function at the current point is calculated, indicating how small changes in the weights affect its output. Thus, the parameters are iteratively adjusted in the opposite direction of the gradient—towards the minima point of the cost curve (see fig. 2.8).

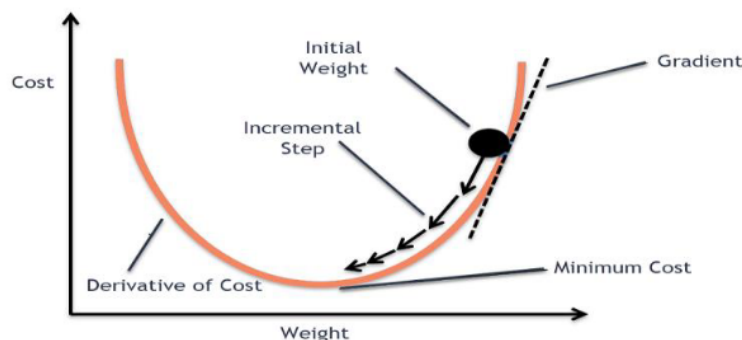


Figure 2.8. Gradient Descent Algorithm[4]

Batch Gradient Descent

Batch Gradient Descent (BGD), also referred to as Vanilla, computes the cost function with regard to the parameters θ for the entire training dataset:

$$\theta_{new} = \theta_{old} - \eta \cdot \nabla_{\theta} J(\theta) \quad (2.4)$$

Here, θ represents the parameters of the function to be minimized, $J(\theta)$ is the loss function, $\nabla_{\theta} J(\theta)$ denotes the gradient of the loss function with respect to θ , and η is the learning rate, which controls the size of the incremental step taken towards the minimum.

Despite being easy to comprehend, BGD requires computing gradients for the entire dataset to update the model once, making it inefficient for very large datasets or those exceeding memory capacity[48]. Additionally, this method does not support online updates with new data as it processes. Nonetheless, batch gradient descent is guaranteed to converge to the global minimum for convex error surfaces and to a local minimum for non-convex surfaces.

Stochastic Gradient Descent

Proceeding with Stochastic Gradient Descent (SGD), this optimization method updates parameters incrementally for each training sample, $x^{(i)}$ and label $y^{(i)}$, unlike the previous discussion regarding BGD.

$$\theta_{new} = \theta_{old} - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)}) \quad (2.5)$$

The term stochastic in the name refers to the random selection of one sample for the gradient calculation instead of the entirety of the dataset. This approach has been proven particularly beneficial for large-scale datasets and online learning algorithms as it aids in reducing training time. Other commonly discussed properties of SGD include the ability to effectively escape local minima, improved model generalizability to unseen data, and the detailed rate of improvement due to the frequency of updates. At the same time, this per-sample frequency of parameter updates also leads to fluctuations in the error rate, instead of a steady and controlled decrease.

Mini-Batch Gradient Descent

This approach of optimization fills the middle space between BGD and SGD, as Mini-Batch Gradient Descent (MBGD) strikes a balance between the computational efficiency of the former and the stability of the latter. Instead of updating parameters using all or only one of the training examples, Mini-Batch Gradient Descent uses a subset of n training data to perform each update:

$$\theta_{new} = \theta_{old} - \eta \cdot \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)}) \quad (2.6)$$

This approach reduces the variance of the parameter updates, which can lead to more stable convergence than SGD while being significantly faster than using the full dataset as in Batch Gradient Descent. This method is particularly effective for training on large datasets making use of highly optimized matrix operations common to state-of-the-art deep learning libraries, resulting in great efficiency while computing the gradients. All the above have contributed greatly to making MBGD the first choice of optimization when training neural networks, while sometimes this is the case even when SGD is reported as the optimization algorithm as the term SGD is used also when mini-batches are employed.

2.3.2 Towards ADAM Optimizer

Learning Rate Selection

The learning rate hyperparameter, often denoted as η , is undoubtedly one of the most crucial factors contributing to the convergence of gradient-based optimization algorithms[48]. It determines the incremental step size of the algorithm towards the minimum of the cost function, as observed in fig. 2.8. The importance of balancing the hyperparameter's value cannot be overstated; too high a learning rate can cause the algorithm to overshoot the minimum, while too low a learning rate can result in excessively slow convergence or getting stuck in local minima (see fig. 2.9). Therefore, achieving balance in this situation suggests swift convergence to the minimum.

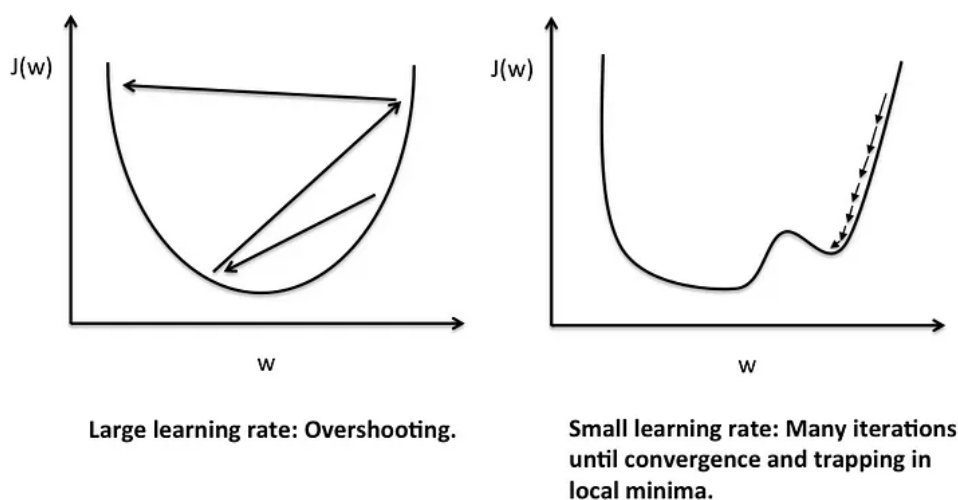


Figure 2.9. The effects of unsuccessful learning rate selection[5]

Momentum

Momentum is an enhancement of Gradient Descent that accelerates convergence, particularly in the context of deep neural networks. The term finds great applicability in situations where the network is not well-conditioned, leading to the composition of *ravines* —surfaces that curve more steeply in one direction than in another direction[49, 4]. Such surfaces negatively impact convergence, due to the simple fact that the gradient does not point towards the minimum, and successive steps of gradient descent can oscillate from one side to the other, progressing, albeit very slowly, to the minimum. Figure 2.10 illustrates how the addition of momentum aids in speeding up convergence to the minimum by damping these oscillations.

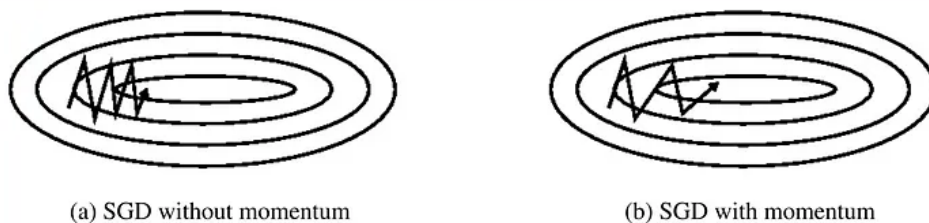


Figure 2.10. Trajectories in ravine surfaces, with and without momentum[6].

In the formal mathematical representation of SGD Momentum in eq. (2.7), v_t represents the velocity, β is the momentum coefficient (typically set between 0.9 and 0.99), and $\nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$ is

the gradient of the loss function. In other words, the velocity that is calculated for a given moment t complements the gradient descent calculation by taking into account the momentum βv_{t-1} . This way, the update on the network's parameters is computed utilizing the velocity term that includes both gradient and momentum.

$$\begin{cases} v_t = \underbrace{\beta v_{t-1}}_{\text{momentum}} - \underbrace{\eta \cdot \nabla_{\partial} J(\partial; x^{(i)}; y^{(i)})}_{\text{gradient descent}} & \triangleright \text{Velocity computation} \\ \partial_t = \partial_{t-1} + v_t & \triangleright \text{Parameters' update} \end{cases} \quad (2.7)$$

ADAM Optimizer

The ADAM (Adaptive Moment Estimation) optimizer, proposed by Kingma and Ba (2017)[50], is a widely utilized algorithm that combines the benefits of both momentum and adaptive learning rates, classifying it as an adaptive learning algorithm. This approach leverages concepts from Adagrad[51] and RMSprop[52], making it particularly effective for deep neural network (DNN) training by managing sparse gradients and non-stationary objectives.

By maintaining exponentially decaying averages, ADAM calculates adaptive learning rates of past gradients and squared gradients. These moving averages smooth out noise in the gradient updates, providing a stable convergence path. The momentum aspect, derived from the average of past gradients, helps accelerate convergence by maintaining consistent update directions, thus reducing oscillations. Its implementation is summarized by algorithm 2.1:

In the following steps, m_t and v_t represent the first moment (mean) and second moment (uncentered variance) of the gradients, respectively. The corrected estimates, $\hat{m}_t$ and $\hat{v}_t$, are used to compute the parameter updates. The decay rates β_1 and β_2 are typically set to 0.9 and 0.999, respectively, with ϵ preventing division by zero.

ADAM's ability to adaptively adjust learning rates for each parameter, based on the mean and variance of gradients, ensures robust performance across various machine learning applications. This is especially beneficial for hyperparameter tuning and managing high-dimensional loss landscapes.

ALGORITHM 2.1: ADAM optimizer algorithm

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\partial)$: Stochastic objective function with parameters ∂

Require: ∂_0 : Initial parameter vector

Initialize:

$m_0 \leftarrow 0$ $\triangleright$ Initialize 1st moment vector
 $v_0 \leftarrow 0$ $\triangleright$ Initialize 2nd moment vector
 $t \leftarrow 0$ $\triangleright$ Initialize timestep

while ∂_t not converged **do**

$t \leftarrow t + 1$
 $g_t \leftarrow \nabla_{\partial} f_t(\partial_{t-1})$ $\triangleright$ Get gradients w.r.t. stochastic objective at timestep t
 $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ $\triangleright$ Update biased first moment estimate
 $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ $\triangleright$ Update biased second raw moment estimate
 $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ $\triangleright$ Compute bias-corrected first moment estimate
 $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ $\triangleright$ Compute bias-corrected second raw moment estimate
 $\partial_t \leftarrow \partial_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ $\triangleright$ Update parameters

end while

return ∂_t $\triangleright$ Resulting parameters

In the Federated Learning realm that is studied, ADAM's optimization techniques are crucial for enhancing convergence rates and model performance. The decentralized nature of Federated Learning introduces challenges such as heterogeneous data distributions and communication constraints, as will be discussed in subsequent sections. ADAM's adaptive learning rate mechanism facilitates effective training across distributed nodes, overcoming data variability and ensuring efficient global model convergence.

2.4 Distributed Machine Learning

2.4.1 The Need for Distribution

As the field of machine learning (ML) continues to evolve, the demand for processing vast amounts of data and training increasingly complex models has surged. Traditional machine-learning methodologies, which typically operate on single-machine setups, face significant limitations in terms of computational power and memory capacity. This led, in the past, to the development and adoption of centralized learning approaches, where data from various sources is aggregated into a central location for processing. While centralized learning mitigates some of the constraints of single-machine learning, it still struggles with issues such as data transfer overhead, security risks, and the inability to scale efficiently with ever-growing datasets.

To overcome these shortcomings, the paradigm has shifted towards Distributed Machine Learning (DML). DML allows for the distribution of data and computational tasks across multiple machines or devices, leveraging their collective resources. This approach not only enhances computational efficiency through parallelism, but also enables the processing of larger datasets that would be infeasible for centralized systems. By distributing the workload, DML restricts data transfer bottlenecks, enhances fault tolerance, and provides greater scalability, making it an essential advancement for modern machine learning applications.

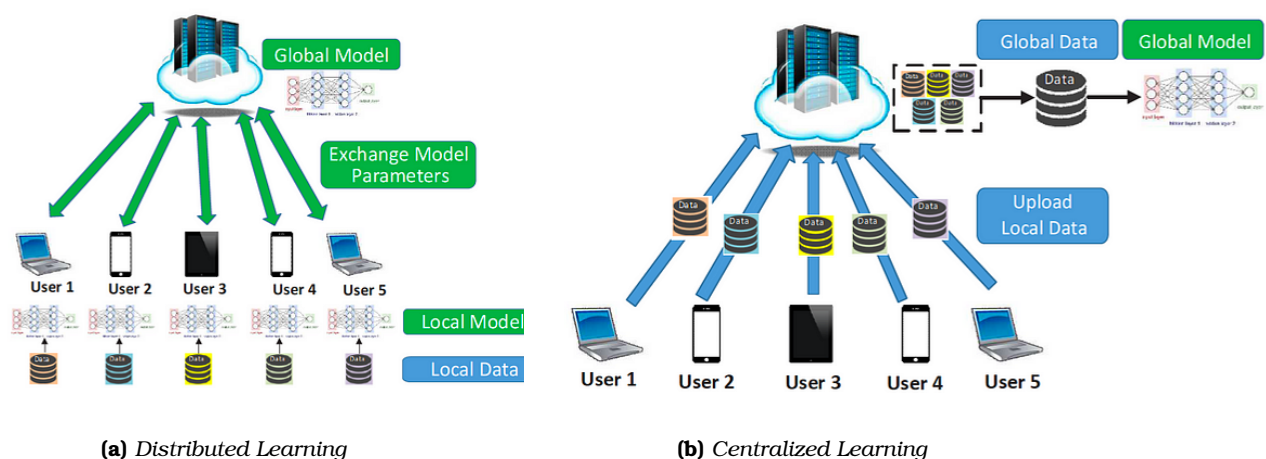


Figure 2.11. A comparison between distributed and centralized learning approaches

Thus, the progression from traditional ML to centralized learning, and finally to DML, reflects the ongoing need to optimize and scale machine learning processes to keep pace with the increasing data demands and computational complexities of contemporary applications.

2.4.2 Distributed Learning Systems Architecture

The architecture of DML Systems can be defined by a triad of layers dependent on each other: *The Distribution Method (Parallelism)*, *the System's Communication Topology*, and *the ML algorithm*. A broad overview of the above will be presented as follows:

Parallelism in Distributed Learning Systems

Parallelism in DML Systems can be achieved with two methods; Data and Model Parallelism. A thorough survey by Joost Verbraeken et al.[53] clarifies:

- ◆ **Data Parallelism:** In the data-parallel approach, the dataset is divided into segments corresponding to the number of worker nodes in the system. Each worker node then processes its respective data segment using the same algorithm. The model is either centralized or replicated across all worker nodes, ensuring a unified output. This method is applicable to any machine learning algorithm that assumes independent and identical distribution (i.i.d.) of data samples, which includes the majority of ML algorithms.
- ◆ **Model Parallelism:** Model parallelism involves splitting the model itself across multiple machines and training different parts of the model on different machines. This approach is useful when the model is too large to fit in the memory of a single machine, or when certain parts of the model require more computation than others. Model parallelism is a bit more complex to implement and is less common than data parallelism, but is still used in some specialized applications, as it cannot automatically be applied to every machine learning algorithm because the model parameters generally cannot be split up.

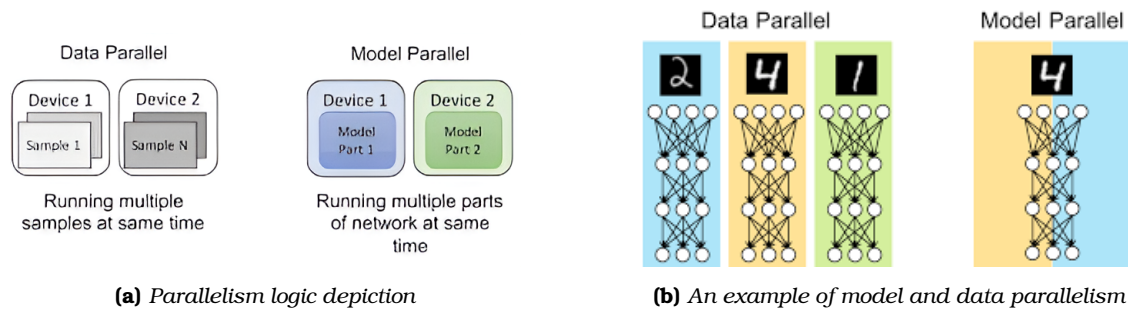


Figure 2.12. Data and Model Parallelism Methods

Distributed Systems Communication Topologies

In DML, the communication architecture delineates the specific roles of training nodes. These roles include computation nodes, which compute gradients, and parameter aggregation nodes, responsible for aggregating these gradients and updating the model parameters. Additionally, it defines the logical topologies for node communication, such as star or ring configurations. Regardless of the model type and learning objective, the same communication architecture facilitates parameter exchange and synchronization when using gradient-based optimization algorithms. A Recent article from Liu et al. [54] showcases the three primarily used connectivities in Distributed Systems.

- ◆ **Ring-All Reduce:** The Ring architecture arranges nodes in a ring topology, where each node communicates directly with its two neighbors. Gradients are shared in a circular manner, with each node sending and receiving data from its adjacent nodes. This architecture efficiently

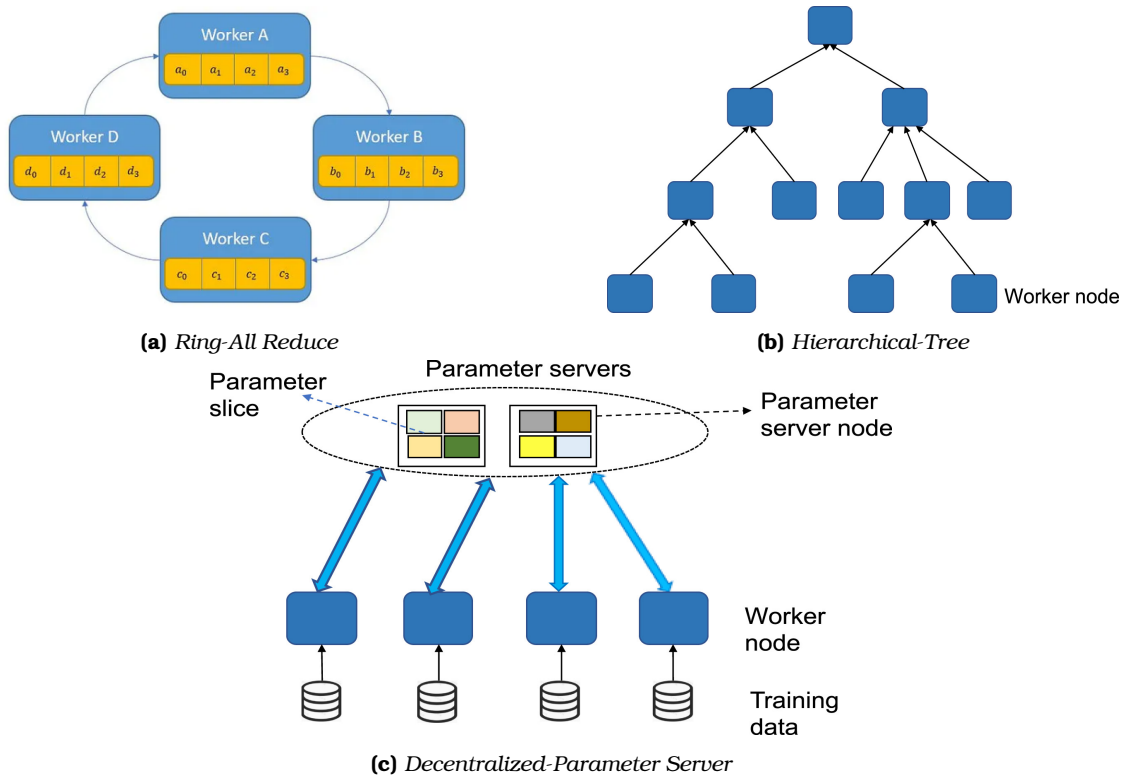


Figure 2.13. Principle Distributed Systems Communication Topologies

utilizes bandwidth by limiting communication to neighboring nodes and avoids the bottlenecks of a centralized coordinator. It is particularly effective in GPU clusters and is widely used in distributed deep learning frameworks. A notable example of this topology is the Ring-All Reduce implementation provided by Uber’s Horovod framework[55].

- ◆ *Hierarchical-Tree*: The Hierarchical-Tree architecture organizes nodes in a tree structure, where intermediate nodes aggregate gradients from their child nodes and pass the aggregated results up the tree to the root node. This reduces the communication overhead compared to a fully connected topology and scales efficiently with the number of nodes. The root node eventually computes the global model update and propagates it back down the tree. This architecture provides a balance between scalability and communication efficiency[56].
- ◆ *Decentralized-Parameter Server*: The Decentralized-Parameter Server architecture involves multiple parameter servers distributing the management of model parameters. Each worker node communicates with one or more parameter servers to read and update parameters during training. This decentralization reduces mitigates bottlenecks and improves fault tolerance compared to a single centralized server. The architecture requires synchronization between parameter servers to ensure model consistency across the network.

This connectivity architecture resembles most the research’s implementation, yet is restricted to a singular server to mitigate cross-server communication challenges⁵. In this scenario, all worker nodes directly communicate with the central server, simplifying network management and reducing latency in communication by having a single aggregation point. However, this also introduces a potential bottleneck at the central server and may suffer from single points of failure.

⁵Parameter server implementation with a singular server resembles a star architecture-all worker nodes directly connected to server

Distributed Learning Algorithms

Last but not least, The DML algorithm selection determines how the learning is conducted after *parallelism method* and *communication topology* are established. The objective in DML algorithms always remains to leverage parallelism and efficient communication to accelerate training and enable the handling of large datasets and models that are infeasible for single-machine setups. Therefore, to achieve their goal, data and computation loads are distributed across several nodes—either being various accelerating devices like GPUs or TPUs or completely different machines[57]. An outline of an algorithm to achieve the above is presented:

1. *Data Preparation*: Partitioning the dataset into subsets that can be distributed across multiple worker nodes, ensuring that data distribution maintains the independent and identical distribution (i.i.d.) property when required.
2. *Model Initialization*: All local models are initialized either locally by each node or centrally by a server and distributed. Depending on the needs, the parameters initialization can be random or with a specific model state.
3. *Local Training*: Each worker node processes its assigned data subset to compute local gradients. Parallelism techniques (data parallelism or model parallelism, see section 2.4.2) are applied to enhance computational efficiency.
4. *Gradient Aggregation & Synchronization*: Gradients computed by each worker are sent either to a server or a peer for aggregation, depending on the topology, to update the global model (see section 2.4.2). Synchronization can be synchronous or asynchronous, affecting the overall training dynamics.
5. *Model Update*: The updated global model is transmitted back to the worker nodes to reinstate the global parameter values to the local model.
6. *Iteration*: Steps 3 to 5 are repeated until the stopping condition is met.

At this point, we should delve a little deeper into the *Gradient Aggregation & Synchronization* part outlined previously, in order to sufficiently cover the principal matter of Synchronization in DML Systems. To be specific, Synchronization can be achieved either in Synchronous or Asynchronous manner in DML systems. Each of these approaches inevitably comes with some merits and some challenges. In an effort to highlight those, the fundamental optimization algorithms for gradient calculation, both in Synchronous and Asynchronous Synchronization—namely Synchronous SGD (SSGD) and Asynchronous SGD (ASGD), will be briefly demonstrated.

Synchronous SGD (SSGD) In SSGD all worker nodes must complete their gradient computations and communicate these gradients before the global model parameters are updated. This ensures consistency across model updates, as all nodes work with the same model parameters in each iteration. However, this approach can lead to inefficiencies due to idle times when faster nodes wait for slower ones, also known as the straggler effect. This can impact scalability, especially in large distributed systems where node performance can vary significantly. On final analysis, the rule to update the parameters, executed by the server at each iteration, can be summarized [58]:

$$w_{t+1} = w_t - \eta \cdot \frac{1}{K} \sum_{i=1}^K \nabla f_{\gamma_{t,i}}(w_t) \quad (2.8)$$

Where in the rule above, η denotes the learning rate, K the number of workers, and $\gamma_{t,i}$ an index chosen uniformly at random by the i -th worker from $1, \dots, n$, representing a selected data

point⁶. Thus, $\nabla f_{\gamma_{t,i}}(w_t)$ is the stochastic gradient estimate from the i -th worker for the $\gamma_{t,i}$ data point (or batch).

Asynchronous SGD (ASGD) ASGD, conversely, allows worker nodes to proceed with their computations and update model parameters independently of other nodes. This reduces idle times and can lead to better utilization of computational resources, thereby enhancing scalability. However, the lack of synchronization can result in inconsistencies, as updates may be based on stale gradients, potentially slowing down the convergence of the model or leading to convergence issues. In Summary, the update rule utilized in ASSD is[59]:

$$w_{t+1} = w_t - \eta \nabla f_{\gamma_{t,i}}(w_{t-\tau_i}) \quad (2.9)$$

Where in the rule above, η denotes the learning rate, $\gamma_{t,i}$ is an index chosen uniformly at random by the i -th worker from $\{1, \dots, n\}$, representing a selected data point, $\nabla f_{\gamma_{t,i}}(w_{t-\tau_i})$ is the stochastic gradient estimate from the i -th worker for the $\gamma_{t,i}$ -th data point, computed at a delayed iteration $t - \tau_i$ due to asynchronous updates.

The choice between Synchronous and Asynchronous SGD is crucial in designing effective DML systems. Synchronous SGD is typically preferred when consistency and simplicity are paramount, and the computational environment is relatively homogeneous, minimizing the straggler effect. On the other hand, Asynchronous SGD is advantageous in heterogeneous environments where reducing idle time and improving resource utilization is critical. The same applies to all the modified and improved variations stemming from these fundamental algorithms that comprise the cornerstone of DML algorithm implementations.

Concluding, as stated at the beginning of this subsection section 2.4.2 the architecture of a DML system can be decomposed into the three layers of *emph*The Distribution Method (Parallelism), *the System's Communication Topology* and *the ML algorithm*. Therefore, researchers and practitioners of the field need always be aware of the characteristics, both merits and challenges, various designing options entail, in order to orchestrate efficient and useful DML Systems that can successfully satisfy the problem requirements.

2.4.3 Challenges in Distributed Machine Learning

Distributed Machine Learning (DML) offers significant advantages in processing large datasets and complex models by leveraging parallelism and distributed resources. However, it also introduces several challenges, some of which are depicted in table 2.3, that can impact efficiency, scalability, consistency, and security. These challenges are critical to understand as they set the stage for the transition to Federated Learning (FL), which addresses several of these issues.

Transition to Federated Learning Federated Learning (FL) builds on the principles of DML but addresses many of its inherent challenges, particularly those related to data privacy, communication overhead, and data heterogeneity. To demonstrate, By preserving the training dataset locally Federated Learning never utilizes the network for private data transmission, negating that way any ill-advised privacy infringements. Moreover, By enabling local training on edge devices and aggregating only the necessary updates at the end of a learning round, FL restricts network utilization and additionally enhances privacy. It should be noted also, that Federated Learning systems do not function under the assumption of i.i.d. data which frequently cannot be stated for other DML system setups. Finally, scalability is partially enhanced due to the overall resctriction

⁶It is common in literature to present the SGD approach of the algorithm interchangeably with the Mini-Batch approach. This convention is also followed for this research's implementation, so clarification was necessary to avoid confusion.

Challenge	Description	Addressed by FL
Communication Overhead	Frequent communication for gradient updates and synchronization can lead to significant overhead, particularly in environments with limited bandwidth or high latency.	Yes
Privacy and Security	Distributed systems are vulnerable to privacy breaches and security attacks, as data is transferred across nodes and potentially exposed to unauthorized access.	Yes
Data Heterogeneity	Data distributed across nodes may not be identically and independently distributed (non-IID), leading to biases and inconsistencies in model training.	Yes
Scalability	Managing communication and synchronization efficiently becomes challenging as the number of nodes increases, potentially incurring high costs.	Partially
Fault Tolerance	Ensuring reliability and fault tolerance in a distributed setting is complex, requiring robust mechanisms to handle node failures and network disruptions.	No
Straggler Effect	In synchronous training, faster nodes must wait for slower ones, leading to idle times and inefficient resource utilization.	No

Table 2.3. *Challenges of Distributed Machine Learning*

of communication overheads, as only model update transmission are executed and no data transfers leading to more system resources being available, then again, this performance improvement cannot be objectively measured and in some cases is negligible.

This transition from DML to FL represents an evolution towards more efficient, scalable, and secure machine learning practices, leveraging distributed resources while mitigating the traditional challenges of DML systems. In summary, the challenges of DML portrayed at table 2.3 highlight the need for more advanced approaches like Federated Learning. FL addresses these issues by enabling decentralized data processing and focusing on privacy-preserving methods, making it a crucial advancement for modern machine learning applications, as will be further illustrated in the next section.

2.5 Federated Learning

In an era where cloud computing is the dominant paradigm for learning, Federated Learning (FL) has the potential to drive significant future changes in the industry. Despite the cloud computing market being dominated by tech giants like Google, Amazon, and Microsoft, Google was the first to recognize the emerging need for a democratized approach to machine learning. Taking a leading role in Federated Learning research, Google introduced the term in 2016 with a seminal paper on efficient Distributed Learning[?]. The significance of Google's approach cannot be overstated, as it eliminates the need for maintaining centralized data centers, allowing cloud resources to be allocated for various other purposes.

The term FL represents a significant shift in how machine learning models are trained and

how data privacy is preserved. As an advanced paradigm within Distributed Machine Learning (DML), FL enables the training of models across multiple decentralized devices or servers holding local data samples, without the need to transfer data to a central server. This approach addresses several critical challenges inherent in traditional centralized machine learning, including data privacy, communication overheads, and the handling of heterogeneous data and systems.

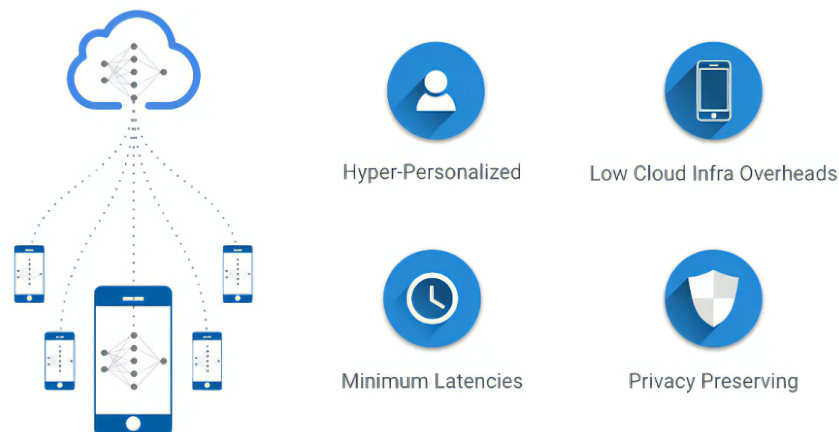


Figure 2.14. *Federated Learning Characteristics.*

2.5.1 Principles of Federated Learning

Providing a solution to these issues FL emerged by enabling model training across decentralized devices or servers while keeping the data local. This method is particularly relevant in scenarios where data privacy is paramount, such as healthcare, finance, and personal devices like smartphones. A variety of FL applications can be located in everyday examples-namely image and video automatic categorization, personalized feeds of news or advertisements, and autocorrect and predictive text (f.i. Google’s GBoard[60]).

The fundamental principles that determine FL’s functionality are categorized as follows[61]:

- ◆ *Local Data Stays Local:* Data generated by devices or organizations remains on the local device, preventing the exposure of sensitive information to external threats or privacy breaches, as it does not need to be uploaded to a centralized server for processing.
- ◆ *Collaborative Model Training:* A central server distributes a global model to devices, which update it based on their local data. These updates, typically in the form of gradients or parameters, are sent back to the server and aggregated to enhance the global model iteratively, thus improving accuracy and utility without accessing the raw data.
- ◆ *Privacy by Design:* FL integrates privacy-preserving mechanisms such as secure multi-party computation and differential privacy, ensuring that model updates shared with the server do not disclose sensitive information about the underlying data or individuals. The privacy rules that are applied, are determined both by international organisms⁷ and local authorities relevant to the region.

⁷Privacy preserving legislations such as the General Data Protection Regulation (GDPR) in the European Union and the California Consumer Privacy Act (CCPA) in the United States[62, 63].

- ◆ **Efficiency and Scalability:** By processing data locally and exchanging only small model updates, FL reduces bandwidth requirements for model training, making it scalable across numerous devices and participants, each contributing to the refinement of the global model.

2.5.2 Federated Learning Categorizations

Several criteria can be used to determine a holistic categorization of Federated Learning (FL) approaches[7]. For the purpose of this research study, the two principal factors are the scale of learning and the alignment of the distributed data. Based on the learning scale, the separation is executed into Cross-Device and Cross-Silo settings. Based on the data distribution, the categorization is between Horizontal and Vertical learning settings. The terms presented above will be explained next:

Nature of the Participants: The distinction is based on whether the federated learning system involves numerous personal or edge devices (cross-device) or a smaller number of institutions or organizations (cross-silo).

Alignment of the Distributed Data: The distinction is based on whether the datasets across entities share the same feature space but have different samples (horizontal) or share the same sample space but have different features (vertical).

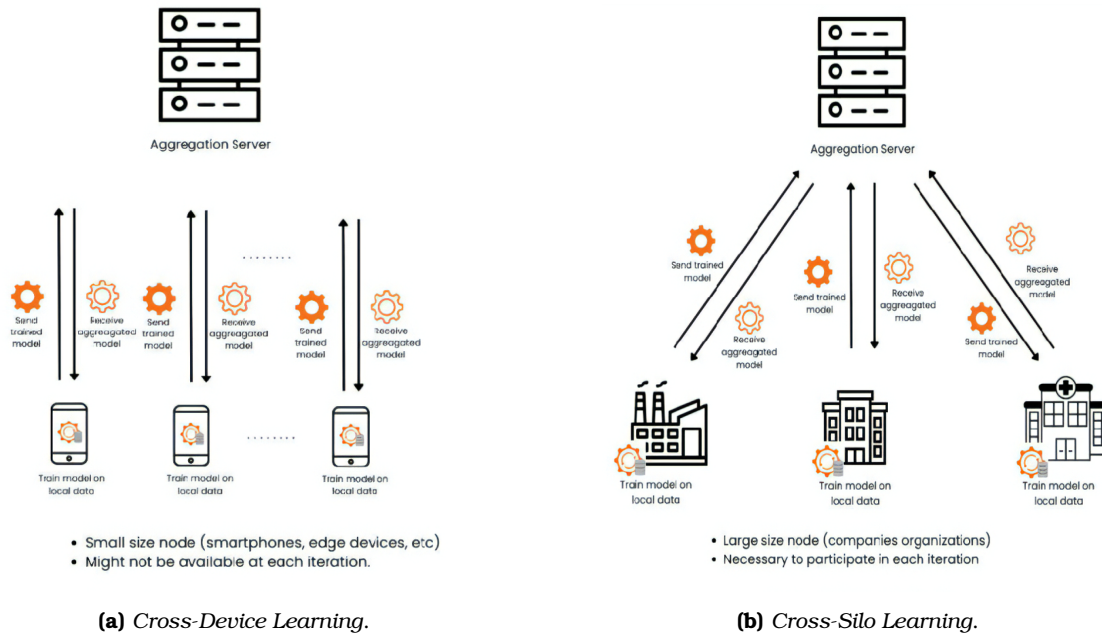


Figure 2.15. Federated Systems separation in Cross-Device or Cross-Silo scale

Cross-Device Federated Learning Cross-device FL involves training models on a large number of edge devices such as smartphones and IoT devices (see fig. 2.15a). Each device holds a small, local dataset. This type of FL is particularly useful for applications where individual user data is sensitive, and thus cannot be shared directly. The primary challenges include managing the diversity in device capabilities, ensuring efficient communication, and maintaining data privacy. Techniques like asynchronous updates and efficient gradient compression are often employed to address these challenges[64, 65]

Cross-Silo Federated Learning Cross-silo FL is used when a small number of organizations, such as hospitals or financial institutions, collaborate to train a machine learning model (see fig. 2.15b). Each organization (silo) has a larger dataset, but data cannot be shared due to privacy concerns. This method often aligns with regulatory requirements such as GDPR and CCPA. Challenges include managing non-IID data distributions across silos and ensuring secure aggregation of model updates. Techniques like secure multi-party computation (SMPC) and differential privacy are crucial in this context[66, 67].

Horizontal and Vertical Federated Learning

Horizontal Federated Learning (HFL) HFL, or sample-based FL, occurs when different entities (devices or institutions) have datasets with the same feature space but different samples. For instance, multiple hospitals might have patient records with the same attributes. HFL is commonly used in both cross-device and cross-silo settings. The main challenge is handling non-IID data, where each client's data distribution may vary significantly. Approaches like data augmentation and federated averaging (FedAvg) are often used to mitigate these issues[68].

Vertical Federated Learning (VFL) VFL, or feature-based FL, occurs when entities have datasets with different feature spaces but the same sample space. For example, one organization might have transaction data, while another has demographic data about the same customers. VFL is typically used in cross-silo settings. The primary challenge is ensuring secure and efficient computation when combining different types of data. Secure computation techniques and feature alignment methods are critical to address these challenges [66, 65].

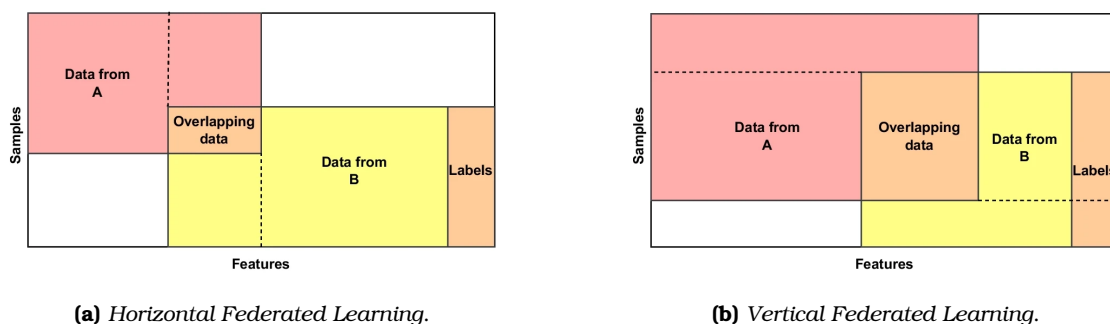


Figure 2.16. Federated Systems separation in horizontal or vertical learning[7].

2.5.3 Federated Learning Methodology

Architecture

In a Federated Learning setup, the architecture can be decomposed into four principal parts: The client devices, the central server, the communication protocols, and the security mechanisms. An overview of each component's functionality will be presented for a typical Federated Learning Architecture based on the cumulative analysis provided by multiple literature sources[64, 66, 68, 69]. It is avoided to delve further into details for some of the presented terms in cases where they are not relevant to the research's objectives. In those circumstances, just a conceptual illustration is provided.

Client Devices: Client devices are the edge devices or local servers that generate and hold the local data. These can include smartphones, IoT devices, or organizational databases. Each client

device performs local training on its dataset, updating the model's parameters based on local data. This helps preserve data privacy since the raw data never leaves the device. After training locally, the clients send their model updates (e.g., gradients or parameters) to the central server. These updates are computed using the local data, ensuring that sensitive or personal information is not exposed. The heterogeneity of these devices, in terms of computational power and network connectivity, presents a challenge that must be managed by the system.

Central Server: The central server acts as a coordinator that aggregates model updates from multiple client devices and updates the global model. Essentially, the server collects model updates sent by the clients, aggregates these updates to refine the global model, and then redistributes the updated global model back to the clients for further training iterations. This process ensures that the global model improves iteratively based on the decentralized data from all participating clients. The central server must handle the challenges of aggregating updates efficiently and securely to maintain the model's performance and integrity.

Communication Protocols: The communication protocol defines the rules and methods for data exchange between client devices and the central server. Efficient communication protocols are essential to minimize latency and bandwidth usage during the model update exchanges. Protocols in FL often incorporate techniques to reduce the volume of data transferred, such as model compression and gradient sparsification. Asynchronous communication methods can handle the varying availability and performance of client devices, reducing overall training time and mitigating the straggler effect.

Security Mechanisms: Security mechanisms are critical in FL to ensure that the data and model updates remain confidential and tamper-proof. These mechanisms protect against potential attacks that could compromise the privacy and integrity of the data being processed. Common security measures include encryption, differential privacy, and secure multi-party computation (SMPC). Encryption ensures that data transmitted between clients and the server is protected from eavesdropping. Differential privacy adds noise to the model updates to prevent the reconstruction of sensitive data. SMPC allows multiple parties to compute a function over their inputs while keeping those inputs private, further enhancing the security of the federated learning process.

2.5.4 Problem Formulation in Federated Learning

This section provides a clear overview of the FL problem statement based on the Almanifi et al. article (2023)[8] observations. In the broader field of Machine Learning (ML), the primary problem is to learn a model that maps a set of input data x to a set of desired outputs y based on a number of training examples. The performance of this model is evaluated using a loss function $f_i(w)$, which measures the error between the model's predictions and the actual desired outputs. For a set of n input-output pairs $\{x_i, y_i\}_{i=1}^n$, the goal is to find the optimal weight vector(s) w that minimizes the loss function. An example of this in a linear regression problem is:

$$f_i(w) = \frac{1}{2}(x_i^T w - y_i)^2, \quad y_i \in \mathbb{R} \text{ and } x_i \in \mathbb{R}^d \quad (2.10)$$

Although loss functions vary across different ML techniques, the problem generally can be formulated as an optimization problem for a finite-sum of functions, expressed as:

$$\min_{w \in \mathbb{R}^d} f(w) \quad \text{where} \quad f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w) \quad (2.11)$$

In Federated Learning, the training process minimizes the loss function locally through gradient descent before the weights w are communicated to the central server. Consider an FL system with a fixed number of clients K , where each client hosts a partition of the data n_k , indexed by $\mathcal{P}_k = \{1, 2, \dots, i\}$. Upon receiving all local parameters, the global server aggregates w to update the new global model. The aggregation mechanism sets the global objective. For example, in FedAVG, one of the most commonly used aggregation methods in FL, the loss function is updated to:

$$f(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w) \quad \text{where} \quad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} f_i(w) \quad (2.12)$$

Thus, the problem in FL becomes learning the aforementioned model in a distributed manner without exchanging raw data.

Federated Learning Process

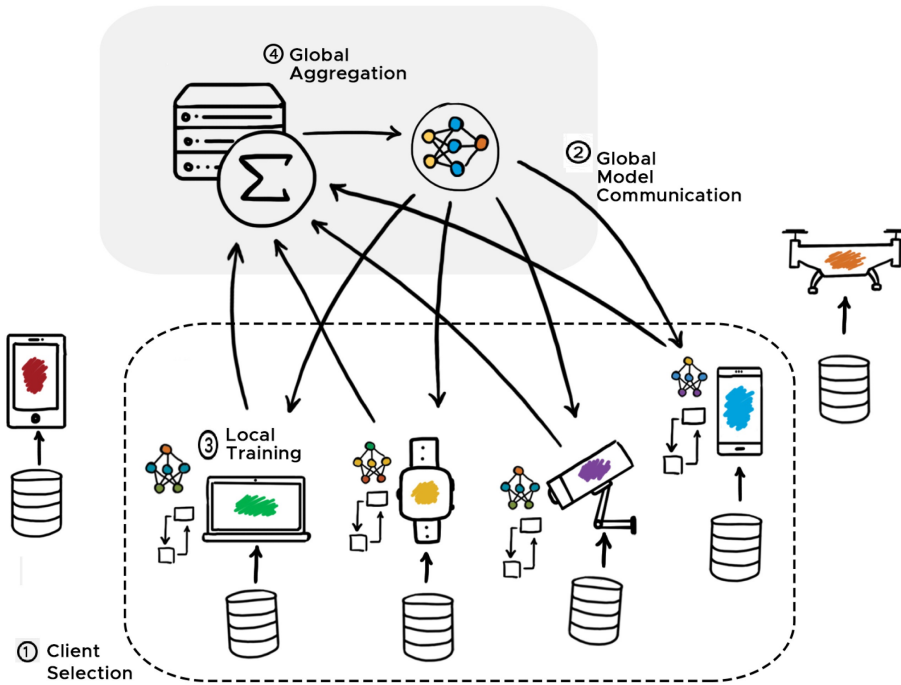


Figure 2.17. General FL training process[8].

In distributed machine learning setups, such as Federated Learning, processes are often viewed as continuous, meaning that the system continuously updates and refines the model parameters based on incoming data. This continuous process allows for dynamic adjustments and real-time improvements, making the system more adaptive and responsive to changes in the data[70, 8]. Provided that, in the iterative training process, each iteration/round can be separated into four major steps as displayed in fig. 2.17:

1. *Client Selection:* Selection strategies vary, as client selection can be executed either randomly, or with prioritization criteria in mind. Depending the FL setup, client selection can be conducted at two different stages: either before sending the global model to the clients (first step) or after local training (third step). *First Step Selection:* The central manager selects a subset of clients before sending the global model. This pre-selection can be used to restrict the number of clients that participate in training, while also gives the opportunity to filter clients based on their update quality. *Third Step Selection:* After local training, the server selects a subset of

clients to transmit their model updates. This step prevents diminishing returns seen when the full set of clients is used. In our implementation client selection is conducted before the global model communication.

2. *Global Model Communication:* The process begins with available clients notifying the server of their readiness to participate. The server then communicates the global model to all participating clients, signaling the beginning of the communication round. This step involves broadcasting the global model's parameters, which include weights and biases, along with any relevant metadata such as hyperparameters (e.g., learning rate) and other arguments (e.g., batch size).
3. *Local Training:* Each client trains the model locally using its own local dataset. During this step, the clients adjust the global model parameters based on their local data, resulting in a *Model Update*. The training process is conducted according to the instructions and hyperparameters provided by the server. The objective function of the training process is notated as f , and for a selected client n from the set of clients N , it attempts to minimize its loss function L , which depends on the global model weights w_g and the local data d_i :

$$f_n(w_g) = \arg \min_{n \in N} L(w_g; d_n) \quad (2.13)$$

After training, clients need to transmit their model updates back to the central server for aggregation (see item 4). The model updates may undergo additional processing to enhance efficiency and security, as outlined in section 2.5.3. Finally, the processed model updates are transmitted to the server, and the clients remain inactive until selected again.

4. *Global Aggregation:* Upon receiving the updated parameters from each selected client, the server aggregates them into a global model using a chosen aggregation mechanism. Various strategies can be employed, such as the FedAVG method, which averages the updates from the clients. The global objective is to minimize the overall loss function as shown in eq. (2.11). Once the model is updated, the communication round ends. The server then evaluates the global model's performance using metrics over a public test set. Based on this evaluation, training may be terminated if the desired accuracy is achieved or the preset number of rounds is completed; otherwise, a new communication round begins.

The above functionality describes a synchronous federated learning setup, which aligns with the thesis implementation. However, as highlighted earlier in order to improve a distributed system's performance, asynchronous methods can be applied too, mitigating the straggler effect.

While programming an asynchronous learning system is taxing and requires careful considerations about out-of-sync updates, the fundamental logic remains the same with modifications primarily pertain to the timing and coordination of these steps. To be exact, In an asynchronous federated learning setup, client selection remains dynamic, but clients are allowed to start and finish their local training at different times. This flexibility means that the server can receive and aggregate updates from clients as they become available, without waiting for all selected clients to complete their training. Consequently, global model communication is also more fluid, with the server sending the global model to clients on an as-needed basis, rather than all at once. Local training on each client still follows the same process, but clients send their updates to the server independently upon completion. The server continuously aggregates these updates using the chosen aggregation mechanism, such as weighted averaging, and updates the global model iteratively.

2.6 Networking in Federated Systems

2.6.1 Network Topologies

The underlying network infrastructure plays a crucial role in the performance and scalability of Federated Learning FL and Distributed Machine Learning systems, in general. Three of the most prevalent topologies employed in DML Systems were described earlier in section 2.4.2. Likewise, it can be stated that, by being a subset of DML systems Federated Systems are constructed using the same fundamental topologies. That said, FL Systems are indeed typically arranged in star, hierarchical tree, ring or peer-to-peer topologies.

Recapitulating briefly in regards with FL, *star topologies*, are a simplification of parameter server setups, where N clients are directly connected with one central server (parameter server) who coordinates the learning procedure. The star topology is advantageous for its simplicity and ease of implementation. However, it can become a bottleneck and a single point of failure, limiting the system's scalability and robustness.

Hierarchical tree topologies, differ from star topologies due to the fact that clients do not always communicate directly with the server. Instead they might be organized in smaller clusters of M clients where $M < N$, that transmit their model updates to intermediate stations responsible for aggregating local model updates, communicating the intermediate aggregation results to the server. This hierarchical approach can improve scalability by distributing the aggregation workload and reducing the communication burden on the central server. Additionally, it can enhance fault tolerance by localizing the impact of network failures.

Ring setups indicate that each node communicates directly with its two neighbors. This way, communication is conducted in a circular manner, presenting a peer-to-peer approximation approach where the coordination responsibility is distributed to the workers. This topology can provide robustness and fault tolerance, as the failure of a single node does not disrupt the entire network. However, it may suffer from higher latency as the number of nodes increases due to the sequential nature of communication.

Finally, In a *fully peer-to-peer* setup, clients communicate directly with each other broadcasting their model updates without a central coordinating authority. Protocols such as gossip algorithms are used to propagate updates throughout the network. This approach can enhance privacy and decentralization but may require more complex coordination mechanisms.

Overall, it becomes obvious that the selection of each of these topologies for the design of a FL system entails certain merits and challenges in regards with the communication efficiency. This fact can be observed directly through network related metrics such as latency, throughput

2.6.2 Communication in Federated Systems

Communication Protocols

The communication protocol defines the rules and methods for data exchange between client devices and the central server. Efficient communication protocols are essential to maximize performance during model exchange procedures. There is a plethora of contemporary technologies with diverse characteristics being utilized in Federated communication, with most primary examples being gRPC, Protocol Buffers (Protobuf), WebSockets, and HTTP/2 and TCP Sockets.

- ◆ *gRPC* is a high-performance, open-source Remote Procedure Call (RPC) framework that can run in any environment. It uses HTTP/2 for transport, Protocol Buffers (protobuf) as the interface description language, and provides features such as authentication, load balancing, and more.

The efficient serialization and transport mechanisms of gRPC make it well-suited for federated learning scenarios where low latency and high throughput are crucial.

- ◆ *HTTP/2* is a major revision of the HTTP network protocol, focusing on performance improvements. It introduces features like multiplexing, header compression, and server push. This protocol has become a industry standard due to its unmatched efficiency.
- ◆ *Protobufs (Protocol Buffers)* are a language-neutral, platform-neutral, extensible mechanism for serializing structured data. Protobuf is used to define the structure of the messages-namely model parameters, training configurations, and results exchanged between the server and clients. This ensures efficient and compact communication.
- ◆ *WebSockets* provide a full-duplex communication channel over a single, long-lived TCP connection. They are useful for real-time applications requiring continuous data exchange. Thet makes WebSokcets the first choice when maintaining a persistent connection between the server and clients is beneficial. This can be particularly useful in real-time monitoring and control of the FL process.
- ◆ *TCP Sockets* offer a reliable, connection-oriented communication protocol that ensures data integrity and delivery. TCP handles packet loss, data corruption, and data reordering, making it suitable for scenarios where reliability is paramount. In federated learning, TCP sockets can be used to establish robust connections between the server and clients, ensuring that critical updates are transmitted accurately and efficiently.

Communication Enhancement

The performance of federated learning systems is significantly influenced by the type of connectivity used. Different network types offer distinct advantages and are preferred in various scenarios: - *Ethernet*: Provides high reliability and consistent bandwidth, making it ideal for environments requiring stable, high-speed communication, such as data centers and research facilities where large-scale model training occurs. - *Wi-Fi*: Offers flexibility and mobility, suitable for residential or office settings where devices are not fixed and may move around. It is preferred in scenarios like smart homes or collaborative workspaces where user devices frequently change locations. - *Point-to-Point Connections*: Provide direct and reliable communication paths, preferred in specialized settings such as direct communication between servers in a data center or between two devices in close proximity that require secure and fast data exchange without intermediate routers. - *Cellular Networks (4G/5G)*: Useful for mobile and remote applications where devices are dispersed over large geographical areas. This is ideal for applications like connected vehicles or remote health monitoring systems, where data needs to be collected from widely distributed sources.

Regardless of the selected communication protocol, and connectivity setting, communication overheads can always be reduced by minimizing the size of transmitted data. Techniques such as quantization, compression and efficient encoding can significantly decrease the amount of data exchanged between clients and the server, thus reducing latency and improving overall system efficiency.

2.6.3 Communication Performance Metrics

In FL systems, evaluating communication performance is paramount to understanding the efficiency and effectiveness of distributed learning processes. Several critical metrics are used

to assess communication performance, each contributing to a comprehensive analysis of the system's capabilities and limitations.

Latency (L) is a fundamental metric, representing the time interval required for a message to travel from the source node to the destination node. It is indicative of the system's responsiveness. Mathematically, latency is defined as:

$$L = t_r - t_s$$

where t_s is the time a message is sent, and t_r is the time it is received.

Bandwidth (B) measures the maximum rate at which data can be transmitted over a network connection within a specific time frame, reflecting the network's capacity to handle data transfers. It is given by:

$$B = \frac{D}{T}$$

where D is the total amount of data that can be transferred, and T is the theoretical time taken for the transfer.

Throughput (T) is closely related to bandwidth but focuses on the actual rate of successful data transfer over the network, considering factors such as network congestion and protocol overhead. In other words, the first one calculates the maximum theoretically achieved speed of transfer of the channel, while the second represents the actual calculated value accomplished in an experiment. It is defined as:

$$T = \frac{D_r}{T_r}$$

where D_r is the actual amount of data transferred, and T is the actual time taken for the transfer. **Packet Loss Rate (PLR)** is the percentage of packets lost during transmission, which can significantly impact the performance of FL systems. It is calculated as:

$$PLR = \frac{P_{sent} - P_{received}}{P_{sent}} \times 100\%$$

where P_{sent} is the number of packets sent, and $P_{received}$ is the number of packets received successfully.

Dropout Rate (DR) is a significant metric in federated learning, indicating the percentage of clients that fail to complete the training process due to various reasons such as connectivity issues or resource constraints. It is defined as:

$$DR = \frac{N_{dropout}}{N_{total}} \times 100\%$$

where $N_{dropout}$ is the number of clients that dropped out, and N_{total} is the total number of clients.

The **Computation-Communication Ratio (CCR)** assesses the balance between computational load and communication overhead in federated systems. It is crucial for optimizing performance and resource allocation. CCR is defined as:

$$CCR = \frac{C}{C + C_{comm}}$$

where C is the time spent on computation, and C_{comm} is the time spent on communication.

Network characteristics such as latency, bandwidth, reliability, and connection type significantly influence the performance of FL systems. High latency can delay the synchronization of model updates, while limited bandwidth can restrict the amount of data that can be transmitted in each communication round. Network reliability ensures that updates from clients are consistently received by the server, while the chosen topology affects the overall communication overhead. For instance, using TCP sockets ensures reliable data transfer with error checking, and gRPC facili-

tates efficient remote procedure calls, both of which are critical for maintaining the integrity of the FL process. Model compression techniques are also employed to reduce the size of the transmitted updates, thereby mitigating the impact of limited bandwidth and high latency.

2.6.4 Simulating Real-World Networks

Network simulators play a pivotal role in federated learning (FL) research by enabling researchers to test and evaluate the performance of FL systems under various network conditions. This capability is significant because setting up FL environments with numerous devices in a controlled laboratory setting is challenging. Simulators such as *NS-3*, *OMNeT++*, and *Mininet* can emulate real-world network environments, offering valuable insights into how different network characteristics influence the training process.

Mimicking Real-World Network Conditions

Simulators can accurately replicate several aspects of real-world networks, including:

- ◆ *Network Congestion*: By simulating varying levels of network traffic, researchers can observe how congestion impacts data transmission and model updates in FL systems.
- ◆ *Latency Variability*: Simulators can introduce different latency values to examine how delays in communication affect the synchronization and convergence of models.
- ◆ *Bandwidth Fluctuations*: Emulating changes in available bandwidth helps in understanding the effect on the speed and efficiency of data exchanges between clients and the central server.
- ◆ *Error Rates*: Introducing different error rates across multiple clients can mimic real-life scenarios where devices have varying communication capabilities and reliability.
- ◆ *Network Topologies*: Simulators can model different network topologies, such as star, ring, and hierarchical structures, to study their impact on the performance of FL systems.

Measuring Contributions from Simulations

Through the use of network simulators, researchers can measure several key performance metrics that provide insights into the effectiveness and robustness of FL systems:

- ◆ *Impact on Model Convergence*: By analyzing how network conditions such as latency and bandwidth influence the speed and accuracy of model convergence, researchers can identify bottlenecks and optimize communication strategies.
- ◆ *System Robustness*: Simulating scenarios with multiple clients experiencing varying error rates allows for the assessment of the system's robustness and its ability to handle client dropouts or unreliable connections.
- ◆ *Scalability Analysis*: Researchers can observe how the FL system scales with the number of clients, particularly when clients have different communication capabilities. This helps in understanding the limits and potential optimizations for large-scale deployments.
- ◆ *Topology Influence*: Evaluating the effect of different network topologies on system performance provides insights into the most efficient structures for specific FL applications.

In summary, network simulators are indispensable tools in FL research, enabling the detailed study of various network conditions and their impact on the training process. They help in identifying potential issues and optimizing FL systems to ensure efficient, scalable, and robust performance in real-world deployments.

2.7 Summary & Transition to Related Work

The theoretical background overview provided in this chapter laid the groundwork for highlighting the principal elements, definitions and contemporary challenges relevant to our *Federated Learning* research. We began by exploring the broader concept of *Machine Learning* exploring the fundamental principles and objectives of the learning process, proceeding with *Artificial Neural Networks* and deep learning to establish a basis for understanding complex model architectures and contemporary learning paradigms. Then our analysis took a turn towards *Optimization Techniques*, crucial for enhancing the learning efficiency of sophisticated, multi-parameter model architectures like the ones employed in *Federated Learning* setups.

Subsequently, the discussion focused on *Distributed Learning Systems*, highlighting their significance in managing learning with immense datasets distributed in a multitude of worker nodes, while preserving computation efficiency. This naturally led to an in-depth analysis of *Federated Learning*, which builds on distributed learning principles to enhance data privacy and minimize communication overhead by allowing decentralized data processing and model updates. To conclude, a brief demonstration of *Networking in FL* is provided, studying the impact of different topologies and characteristics in the learning process.

All things considered, this in-depth survey of the existing literature provided all the necessary terminology and background, for demonstrating the proposed ideas of this thesis, outlined in the *introduction*. The following chapter contains, the preliminary work, that consists the fundamental basis for the development and simulation of a Federated Learning procedure, employing an optimized synchronization technique to further mitigate communication overheads, in a unified approach for managing the learning process in a user-friendly framework for Federation, as well as simulating the impact of various network parameters to its performance. Matters discussed in the following chapter include: *datasets used in federation*, *common ANN architectures*, *algorithmic implementation of FL* as well as relevant *synchronization optimizations* to conclude with a *FL simulator* encompassing data management in conjunction with network simulation in a holistic approach.

Chapter 3

Related Work

3.1 Datasets in Federated Learning Research

Datasets comprise a fundamental factor in machine learning, including fields such as Federated Learning. Essentially, they offer training samples along with their corresponding labels, providing the algorithm with the necessary input to be able to extract underlying model patterns, relationships, and representations essential for accurate and efficient generalization. A good dataset for machine learning needs to satisfy the following requirements:

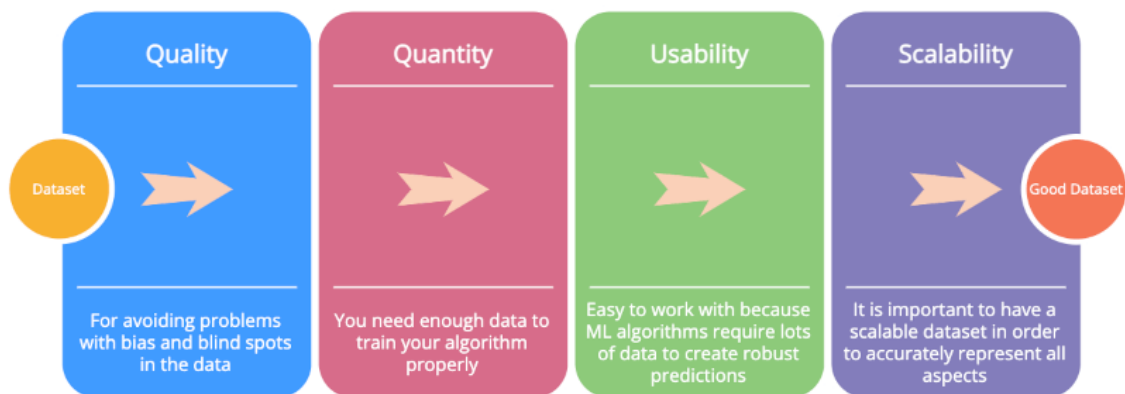


Figure 3.1. Features of a good dataset for machine learning

Quality samples in datasets are accurate, reliable, and contain a balanced distribution among the data classes, ensuring that the model learns from trustworthy and informative examples. Quantity is necessary with ML algorithms requiring a vast number of samples to achieve generalization, offering diverse coverage of the underlying data distributions to avert overfitting and achieve convergence. Usability refers to the dataset being properly formatted and accompanied by clear metadata facilitating seamless utilization by researchers and practitioners. Finally, scalability is required to be able to accommodate varying volumes of data without compromising performance or efficiency.

3.1.1 Benchmark Datasets

The process of creating a proper dataset for machine learning tasks, especially in the context of federated learning, is non-trivial. It involves meticulous curation, preprocessing, and validation to ensure that the dataset accurately represents the underlying data distribution and facilitates meaningful model training. Given the complexity and challenges associated with dataset creation, several benchmark datasets are utilized in the FL community to evaluate the performance of

federated learning algorithms. Some examples relevant to our research’s implementation are presented:

- ♦ **MNIST:** MNIST is a dataset of handwritten digits widely used for image classification tasks. It consists of 28×28 grayscale images of digits from 0 to 9, with corresponding labels.
- ♦ **Fashion-MNIST:** Fashion-MNIST is a dataset of clothing articles used for image classification tasks, similar to MNIST but with more diverse classes. It consists of 28×28 grayscale images of fashion items such as shirts, shoes, and trousers.
- ♦ **CIFAR-10:** CIFAR-10 is a dataset of small images across ten classes, often used for image recognition tasks. It contains 60,000 32×32 color images, with 6,000 images per class.
- ♦ **CIFAR-100:** CIFAR-100 is an extension of CIFAR-10, containing 100 classes of objects. It provides a more challenging classification task compared to CIFAR-10, with finer-grained categories.

These benchmark datasets serve as standardized tasks for evaluating federated learning algorithms, allowing researchers to assess model performance across different domains. By utilizing well-established datasets like MNIST, CIFAR-10, Fashion-MNIST, and CIFAR-100, researchers can compare the effectiveness of their FL implementations, measure their generalization capabilities, and identify areas for improvement in federated learning methodologies. In that spilt, these datasets were preferred in our study.

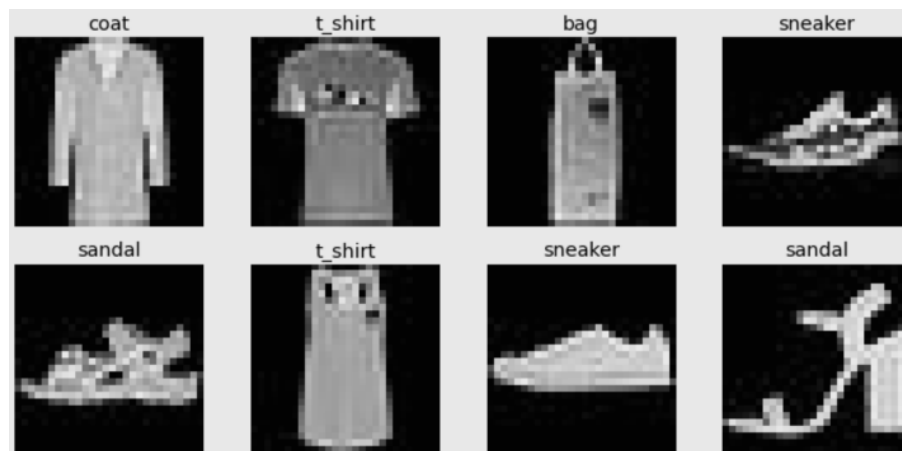


Figure 3.2. Illustration of Fashion-MNIST samples

3.1.2 Real-world Datasets

While benchmark datasets offer controlled environments for evaluating federated learning algorithms, real-world datasets present unique challenges and opportunities, essential for real-life Federated System implementations. Benchmark datasets provide some intuition about system performance but do not encompass fundamental issues characteristic of realistic scenarios, such as heterogeneous distribution, data privacy regulations, and continuous data generation.

As Federated Learning expands into real-life applications in *healthcare*, *mobile apps*, and *finance*, the need for appropriate datasets for these applications increases. Examples include:

- ♦ **Healthcare Data:** Patient records, medical images, and sensor data collected from various healthcare institutions.

- ♦ **IoT Data:** Sensor readings, device logs, and telemetry data generated by Internet-of-Things (IoT) devices.
- ♦ **Financial Data:** Transaction records, credit scores, and market data from banking and financial institutions.

3.1.3 IID VS. non-IID Data Distributions

Two crucial methods of data distribution can be identified in decentralized learning settings.

Independent and Identically Distributed (IID) In an IID data distribution, each client's data is drawn from the same underlying distribution, and the data samples are independent of each other. That is to say, each sample is drawn independently from the same probability distribution and exhibits identical statistical properties with the rest, representing the overall population distribution.

Non-IID Non-IID data distributions occur when each client's data sample follows a different distribution, either due to variations in data sources, data collection methods, or client-specific characteristics.

In many traditional *distributed machine learning systems* research, IID data distributions are frequently assumed, where the central server decentralizes data in such a manner that each client possesses a representative subset of the overall dataset. However, this assumption often overlooks the inherent complexities of real-world data distributions, especially in federated systems. In federated learning, where data remains decentralized across multiple clients, achieving true IID data distributions is often impractical or unrealistic.

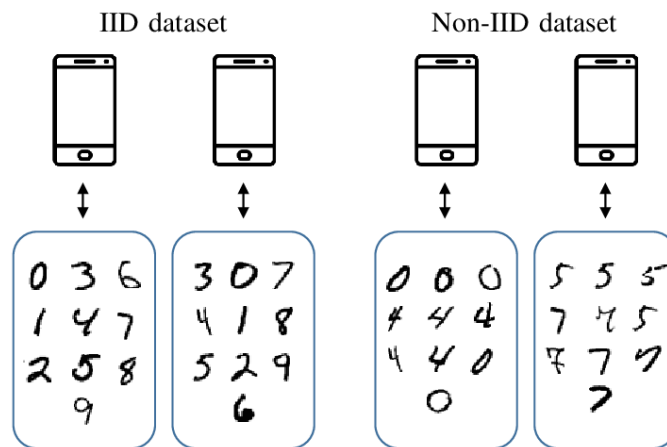


Figure 3.3. Example of IID and non-IID distributions of MNIST: In the IID dataset, each device contains a similar and random sample of digits (0 – 9), ensuring that the data distribution on each device is similar to the overall distribution. In the Non-IID dataset, each device contains digits that are not uniformly distributed; instead, each device has a concentration of certain digits. For example, one device has mostly 0s, another mostly 7s, and so on

In federated learning, data distributions across clients can significantly impact the training process and model performance, thus, both cases should be studied. IID distributions provide a simplified benchmark for evaluating federated learning algorithms, while non-IID reflect the inherent heterogeneity in practical deployments. The conjunction of the two, in conclusion, ensures that federated learning algorithms are not only theoretically sound but also practical and effective in real-world settings.

3.2 ANN Architectures

The never-ending advancement of ANN architectures, has resulted in remarkable progress in image classification, utilizing innovative, and advanced CNNs. Over the years, various ground-breaking models have been introduced, each implementing novel techniques to enhance deep learning performance. The above statement can be additionally verified by the outstanding results in regards with the top-5 error rate accomplished in competitions such as ImageNet's Large Scale Visual Recognition Challenge (ILSVRC) where the error rate plummeted to less than 7% even from 2014[71]. In this section, the aim is to present the primary examples that have paved the way for these improvements from the inspiration startpoint of *LeNet-5* to 2014 and 2015 ILSVRC competition standouts in *VGG16* and *ResNet* respectively.

3.2.1 LeNet-5

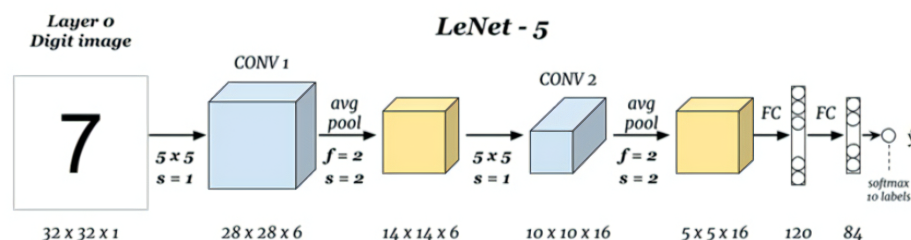


Figure 3.4. *LeNet's 5 Architecture.*

LeNet-5, developed by Yann LeCun and his colleagues in the late 1990s[72], is widely regarded as the pioneer of modern Convolutional Neural Networks. Designed primarily for handwritten digit recognition, specifically the MNIST dataset, LeNet-5 laid the foundation for subsequent advancements in deep learning. The architecture consists of two sets of convolutional and subsampling layers, followed by three fully connected layers. Observing fig. 3.4, the original LeNet architecture utilized average pooling, that was a standard of the era, yet with current knowledge max pooling layers would be preferred, producing slightly improved results. However, for this analysis the vanilla version, as introduced in the original paper, is showcased.

The simple yet effective design of *LeNet-5* demonstrated the potential of CNNs in automatically learning hierarchical features from raw pixel data, a concept that has been fundamental to later architectures. LeNet's success in recognizing handwritten digits was revolutionary for its era, and is still regarded as one of the most important historically CNN architectures.

3.2.2 VGG16

VGG16, introduced by the Visual Geometry Group and specifically A. Zisserman and K. Simonyan from the University of Oxford[73], and based its implementation on the notion that the performance of a deep neural network can be improved by increasing the model's depth. To be precise, the number 16 on the ann's names was assigned for the 16 parameter layers that comprise the model, 13 of which are convolutional layers and the rest 3 are fully connected layers. VGG16 is renowned for its simplicity and robust performance in a variety of computer vision tasks. Its architecture comprises of stacked convolution layers followed by max-pooling layers, with progressively increasing depth.

The primary novelty of the model's architecture was that it mitigates the network's parameters by leveraging 3×3 kernels with a stride of 1 to replicate a kernel of any size. In essence, the

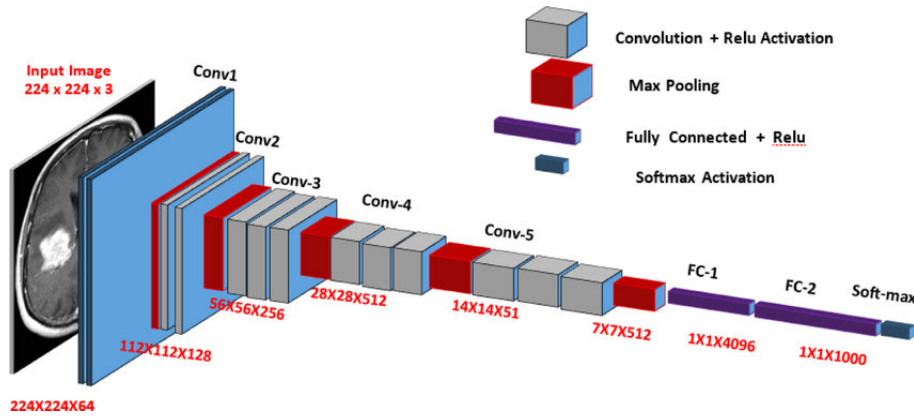


Figure 3.5. VGG16 Architecture.

dimensions of the output of any convolved layer can be reached by stacking several smaller layers. An example will be demonstrated to understand the above logic. Consider a 5×5 image that is convolved with a 5×5 kernel. This produces an output matrix of 1×1 . Let the same image be convolved by a 3×3 kernel, producing a 3×3 matrix that can be additionally convolved by another 3×3 kernel. The result will be an output matrix of 1×1 . In this example both processes result on a 1×1 matrix, yet the single kernel required $5 \cdot 5 = 25$ parameters while the two stacked convolution layers required: $3 \cdot 3 + 3 \cdot 3 = 18$ parameters.

Following the innovative example of AlexNet-the 2012 ILSVRC winner- VGG16 employed ReLU activation functions in all the layers, reducing training time, while leveraging SoftMax function in the output layer to normalize the probability of the classes.

Combining the aforementioned elements, this architecture managed to classify with great precision images to 1000 object categories and became a significant paradigm in the field of ConvNets, still being utilized in systems that conduct deep learning.

3.2.3 ResNet

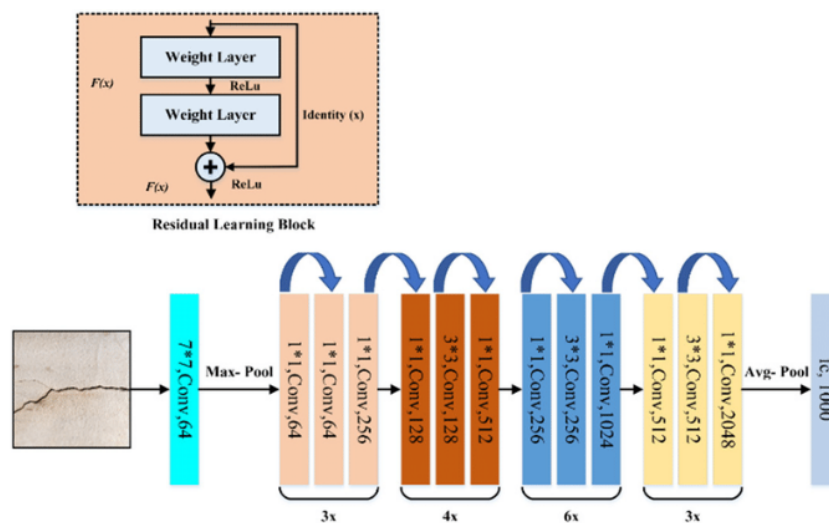


Figure 3.6. ResNet's Architecture with residual learning block illustration.

Residual Networks (ResNet), introduced by Kaiming He et al. in 2015[74], addressed the degradation problem in very deep networks. ResNet's key innovation is the introduction of residual

learning blocks, or shortcuts, which add the input of the block directly to its output, allowing the network to learn identity mappings. These connections help prevent the vanishing gradient phenomenon during backpropagation and mitigate accuracy saturation, enabling the effective training of much deeper networks compared to previous architectures.

ResNet variants, such as ResNet-50, ResNet-101, and ResNet-152, have achieved remarkable success, with ResNet-152 achieving a top-5 error rate of 3.57% in the ILSVRC 2015, setting new performance standards. The success of ResNet demonstrated that increasing network depth could lead to better performance, provided that appropriate architectural adjustments are made to ensure effective training.

These architectures—LeNet, AlexNet, and ResNet—represent significant milestones in the evolution of CNNs, each contributing foundational ideas and techniques that have driven the field forward.

3.3 Federated Learning Algorithms

In this section, we delve into the foundational algorithmic implementation that has shaped Federated Learning research, by introducing a structural background for subsequent efforts. This, algorithm is none other than *FedAvg*, which introduced several key innovations, that contributed as the basis for more sophisticated optimized approaches that are currently employed in the domain. Nonetheless, Before exploring the novelties of FedAvg, we will first discuss FedSGD, as the simplest Federated Learning Algorithm that set the stage for the FedAvg deployment. Concluding the analysis of the two principal algorithms, the shortcomings of contemporary FL implementations are highlighted.

Before focusing on the algorithmic implementations a common notation ground is laid with the table 3.1:

Notation	Definition
w_t	Model weights on communication round # t
w_t^k	Model weights on communication round # t on client k
K	Number of all clients
C	Fraction of clients performing computations in each round
E	Number of training passes each client makes over its local dataset on each round
B	The local minibatch size used for the client updates
η	The learning rate
$\mathcal{P}_k$	Set of data points on client k
n_k	Number of data points on client k
$f_i(w)$	Loss $l(x_i, y_i; w)$ i.e., loss on example (x_i, y_i) with model parameters w

Table 3.1. Notation used in Federated Learning algorithms

3.3.1 Federated Stochastic Gradient Descent (FedSGD)

Stochastic Gradient Descent (SGD), has shown great results in deep learning, making it a foundational algorithm for many optimization problems. Given its effectiveness, researchers decided to base the Federated Learning training algorithm on SGD as a baseline. In traditional Stochastic Gradient Descent, gradients are computed on a random subset of the total dataset and then used to update the model parameters. This process is repeated iteratively, making it computationally efficient for centralized learning.

However, applying SGD naively to the federated optimization problem involves a single batch gradient calculation on a randomly selected client per round of communication. This way each selected client participates in training using the local data points. While this approach is computationally efficient, it requires a large number of communication rounds to achieve good model performance. This is due to the fact that each client only contributes a small fraction of the data in each round.

Federated Stochastic Gradient Descent is a direct adaptation of the traditional SGD algorithm¹ to the federated setting. In the FedSGD process, the server starts by initializing the global model weights w_t . For each communication round t , a fraction C of clients is randomly selected to participate. Each selected client k uses its local dataset $\mathcal{P}_k$ to compute the gradient of the loss function $f_i(w)$ over all its data points n_k . These gradients G_k are then sent to the central server, where they are aggregated. The server updates the global model weights w_t by taking a weighted average of the received gradients, with weights proportional to the number of data points n_k on each client. The updated global model w_{t+1} is then distributed back to all clients, and the process repeats until the stopping criterion is met.

ALGORITHM 3.1: *Federated Stochastic Gradient Descent (FedSGD)*

```

1: Server Execution:
2: Initialize global model weights  $w_0$ , and learning rate  $\eta$ 
3: for each round  $t = 1, 2, \dots$  do
4:    $m \leftarrow \max(C \cdot K, 1)$ , number of participating clients
5:    $S_t \leftarrow$  random set of  $m$  clients
6:   Send global model  $w_t$  to  $S_t$  clients
7:   for each client  $k \in S_t$  in parallel do
8:      $g_k \leftarrow \text{GRADIENTSTEP}(k, w_t)$ 
9:   end for
10:   $w_{t+1} \leftarrow w_t - \eta \sum_{k \in S_t} \frac{n_k}{n} g_k$ 
11:  Send the model  $w_{t+1}$  to all participants
12: end for

13: Client Execution:
14: procedure GRADIENTSTEP( $k, w_t$ )
15:    $g \leftarrow \nabla f_i(w_t)$  over  $\mathcal{P}_k$ 
16:   return  $g$  to server
17: end procedure

```

3.3.2 Federated Averaging (FedAvg)

While FedSGD provides a straightforward extension of SGD to the federated setting, it requires frequent communication between clients and the server, which can be a significant bottleneck in environments with limited bandwidth. To address this issue, the Federated Averaging (FedAvg) algorithm was proposed, introducing several key innovations to reduce communication overhead while maintaining model performance.

FedAvg is a generalization of the FedSGD approach by allowing clients to perform multiple local updates using their own data before communicating with the server. Essentially, this means that each client performs a local gradient descent step upon the his model using local data. After a number of local gradient updates then clients communicate their model updates instead of the single gradient that is, subsequently aggregated by the server. Therefore, not only the frequency of communication rounds is reduced, but also local computations are leveraged more effectively,

¹Or if we want to be precise, it is a direct adaptation of Mini-Batch Gradient Descent

especially if we take into account the forever increasing performance of contemporary IoT devices utilized as clients.

ALGORITHM 3.2: *FederatedAveraging*

```

1: Server executes:
2: Initialize global model weights  $w_0$ , and learning rate  $\eta$ 
3: for each round  $t = 1, 2, \dots$  do do
4:    $m \leftarrow \max(C \cdot K, 1)$ 
5:    $S_t \leftarrow$  random set of  $m$  clients
6:   Send global model  $w_t$  to  $S_t$  clients
7:   for each client  $k \in S_t$  in parallel do do
8:      $w_t^k \leftarrow \text{CLIENTUPDATE}(k, w_t)$ 
9:   end for
10:   $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_t^k$ 
11: end for

12: ClientUpdate( $k, w_t$ ):
13:  $B \leftarrow$  split  $\mathcal{P}_k$  into batches of size  $B$ 
14: for each local epoch  $i$  from 1 to  $E$  do do
15:   for batch  $b \in B$  do do
16:      $w \leftarrow w - \eta \nabla \ell(w; b)$ 
17:   end for
18: end for
19: return  $w$  to server

```

All in all, the amount of computation can be adjusted using three key learning parameters: C , the fraction of clients participating in each round; E , the number of local epochs or training passes each client performs over its local dataset per round; and B , the local minibatch size used for client updates. In the extreme case where $B \rightarrow \infty$ and $E = 1$, FedAvg naturally simplifies to FedSGD, thereby concealing the unique novelties of FedAvg.

The novelties introduced by FedAvg, briefly outlined above, made the algorithm a more practical and efficient approach to federated learning, addressing some of the key limitations of FedSGD. To sum up, these innovations include reduced communication frequency, local computation utilization, and improved scalability. By performing multiple local updates before sending the model to the server, FedAvg significantly reduces the number of communication rounds required. Concurrently, clients leverage their local computational resources more effectively, due to the increasing processing power of modern edge devices. Finally, by taking load out of the server with the local updates, this provides the foundations for enhanced scalability as the processing load is distributed.

3.3.3 Contemporary Challenges of Federated Learning

Federated Learning (FL) faces several contemporary challenges that need to be addressed to enhance its implementation and efficiency. The objective of this research was, among else, to provide a pathway on how to overcome some of these issues, yet the entirety of them would be impossible to be addressed solely with one thesis dissertation. In this spirit, to be able to illustrate succinctly the various shortcomings of Federated Learning, whilst highlighting their relevance to this research study, an outline of the principal challenges, along with insights into potential solutions that are presented in our implementation will be presented. The following analysis is based on the journal article introduced by Li et al.[75] (2020):

Complex infrastructure and Management: Federated Learning demands a robust and complex infrastructure for coordinating numerous distributed devices with diverse hardware, software, and network conditions, making setup and maintenance challenging. Ensuring seamless synchronization, managing device failures, and handling system heterogeneity require sophisticated solutions. Additionally, secure data transmission and efficient resource allocation are essential. Despite advances in machine learning infrastructure and research, the complexity of FL infrastructure reveals a need for simplified deployment and management interfaces. This gap highlights the necessity for easy federation of machine learning processes, making FL more accessible and manageable.

Expensive Communication: Communication Overhead is a critical bottleneck in Federated Networks, that necessitate against transmitting locally generated data. The reality in Federated Learning Setups differs significantly from the almost perfect communication setups of traditional Centralized Learning Data centers, where stable and incredibly fast connections are assumed with restricted failures. In contrast, Federated Learning is conducted on a direct opposite manner, meaning that learning is based on utilizing a vast number of devices, like smartphones, resulting in substantially reduced communication capabilities with several limiting factors including bandwidth energy and transmission power. Therefore, in edge FL, where the datasets are small and the connections are weak and unreliable the communication to computation ratio skyrockets.

System Heterogeneity: Devices in federated networks exhibit significant variability in storage, computational power, and communication capabilities due to differences in hardware (CPU, memory), network connectivity (3G, 4G, 5G, Wi-Fi), and power levels (battery). Typically, only a small fraction of devices are active at any given time, and devices may drop out due to connectivity or energy constraints. These system-level disparities aggravate challenges such as managing stragglers, who delay updates, and ensuring fault tolerance within the network. Robust methods are needed to handle low participation and heterogeneous environments effectively.

Statistical Data Heterogeneity: The need for independent and identically distributed (i.i.d.) datasets is crucial for efficiency in Distributed Systems. In Federated Systems, while non-i.i.d. data can be used, it significantly impacts the speed and difficulty of convergence. Each client trains on its local dataset, adding complexity and likely resulting in skewed updates based on user preferences. The server, due to privacy concerns, cannot know the dataset distribution, necessitating sophisticated approaches to maintain accuracy while learning from highly biased datasets. These methods must effectively address the challenges posed by data heterogeneity to ensure efficient federated learning.

Privacy and Security: The main objective of Federated Learning is to ensure client privacy. However, shared parameters can be exploited by malicious participants or third parties to extract sensitive information, undermining this goal. It is commonly assumed that all clients act in good faith, but some may be malicious, attempting to compromise the model's integrity. They can perform model poisoning or data poisoning attacks, using corrupted data to degrade model accuracy or introduce backdoors. Therefore, robust security measures are crucial to maintain the integrity and privacy of the FL process.

3.3.4 Countermeasures Introduced in Thesis

This thesis addresses several key challenges in Federated Learning, some as primary objectives and others as secondary considerations. The following discussion is organized by relevance to

the thesis contents, starting with the main objectives: mitigating communication overhead and providing a user-friendly interface for FL experimentation.

To reduce communication overhead in FL, this work employs strategies such as minimizing the size of transmitted messages, reducing the number of communication rounds, and limiting communication frequency. Specifically, the thesis implements the Functional Dynamic Averaging (FDA) synchronization technique (section 3.4) to enhance convergence speed, effectively balancing the trade-offs between communication and computation. Additionally, the Flower Framework (section 4.2) is utilized to facilitate a seamless transition from traditional ML frameworks to FL setups, offering an accessible and efficient infrastructure.

While the issue of system heterogeneity is not directly addressed, the thesis integrates a Federated Learning monitoring framework with the Ns3 network simulator (section 4.3). This integration allows for experimentation with various communication setups and testing different device models with diverse characteristics (e.g., power consumption, computational capabilities). Monitoring data heterogeneity is also part of the framework to assess system performance under different conditions.

Privacy and security concerns, although not directly tackled in this study, are acknowledged as critical areas for future research. The experiments are conducted in a controlled environment, which minimizes the immediate need for additional privacy measures. However, addressing these concerns remains essential for the advancement of Federated Learning.

3.4 Synchronization Techniques

Implementing a Federated Learning (FL) system presents unique synchronization challenges, essential for maintaining model accuracy and consistency across distributed nodes. Synchronization is not only critical for synchronous learning tasks, where synchronized model distribution and aggregation are mandatory (see section 2.5.4), but also for any multi-threaded process, including both synchronous and asynchronous setups. Effective synchronization in FL is vital to coordinate the operations of multiple distributed devices, handle concurrent data access, and ensure the consistency and accuracy of the global model. This research emphasizes synchronized scenarios, necessitating careful considerations to manage model aggregation and ensure thread safety. To achieve this, various synchronization techniques are employed, focusing on both synchronous federated learning methods and traditional synchronization primitives used in multiprogramming environments.

3.4.1 Geometric Method

The Geometric Method comprises an innovative approach to effectively overseeing threshold functions across distributed data streams introduced by Sharfman et al. (2010)[9] with their research "A Geometric Approach to Monitoring Threshold Functions over Distributed Data Streams". The primary motivation of the proposed method is to mitigate the communication costs entailed by frequent interprocess communication— a common characteristic of distributed learning processes. In previous sections, it was highlighted that in order to tackle communication overheads, the two major approaches are either to restrict the size of the communicated information or to minimize the frequency of communication. The geometric reformulation of the issue at hand, drastically reduces the need for communication in distributed scenarios, enlisting the aforementioned technique under the latter category.

The communication frequency restriction is accomplished through the decomposition of the global monitoring task into a set of localized constraints, derived from the geometric properties

of the problem. In this manner, worker nodes are enabled to individually monitor their data streams, utilizing outsourced communication with the parameter server or other workers solely on the occasion that these local constraints are violated. Such violations indicate that the local data changes are significant enough to potentially affect the global monitored function, while irrelevant data changes are filtered out.

Overview of Implementation:

Having established a foundational basis of the general concept introduced by the geometric method, at this point, we will delve further into some key aspects regarding the implementation of the underlying logic. Consider a network comprised by n nodes and $\mathbb{S} = \{s_1, s_2, \dots, s_n\}$ being an arbitrary set of data streams collected by nodes of $\mathbb{P} = \{p_1, p_2, \dots, p_n\}$ representing the set of nodes. Then we can define two types of vectors crucial for the analysis:

$$\begin{cases} u_1(t), u_2(t), \dots, u_n(t) & : \text{Local statistics vectors} \\ u(t) = \frac{\sum_{i=1}^n w_i u_i(t)}{\sum_{i=1}^n w_i} & : \text{Global statistics vector} \end{cases} \quad (3.1)$$

Where in the above statements $u_i(t) \in \mathbb{R}^d$, represents the local statistics of node p_i and w_i is a positive weight assigned to node p_i , usually corresponding to the population of elements in the local statistics of the node. Combined in the second equation we get the weighted average of local statistics producing a global illustration.

Nonetheless, in order to maintain the value of the *global statistics* ($u(t)$), at each time t the *local statistics vectors* ($u_i(t)$) would need to be transmitted, resulting in expensive communications. Thus, another vector is introduced denoted as $e(t)$ that represents an estimate of the global statistics and is computed by the *local statistics vectors* collected at certain times. This *estimate vector* is known at any time by all the nodes. Before presenting the mathematical definition u'_i needs to be introduced, which consists the last statistics vector collected by the node p_i :

$$e(t) = \frac{\sum_{i=1}^n w_i u'_i}{\sum_{i=1}^n w_i}$$

Therefore the estimate vector essentially is the weighted average of the latest statistics vectors collected from the nodes. Finalizing with the notation: the *statistics delta vector*: $\Delta u_i(t)$ and the *drift vector*: $v_i(t)$ are the last essential vectors to define:

$$\begin{cases} \Delta u_i(t) = u_i(t) - u'_i & : \text{Statistics delta vector} \\ v_i(t) = e(t) + \Delta u_i(t) & : \text{Drift vector} \end{cases} \quad (3.2)$$

The first portrays the difference between the current local statistics vector and the last statistics vector transmitted and the second displacement of $\Delta u_i(t)$ in relation to the estimate vector.

It is important to note at this point that the geometric method finds application in two types of environments. That way the implementation branches in a decentralized setting, where nodes can efficiently broadcast messages, and a coordinator-based setting, where the broadcasting cost is high. The variables previously defined are relevant for both setups, however, but the process slightly differs. At this point, an overview of the algorithm will be provided in both setups to attain a better illustration of the process. Subsequently, the undelaying logic will be displayed to form an understanding of the geometric method.

Decentralized Setting:

In the decentralized setting, when the algorithm dictates that a node's local statistics vector needs to be transmitted, meaning it is important to update the estimate vector to maintain accuracy, then that same node broadcasts his local statistics vector to the rest of the nodes. In this type of setting, all nodes retain in memory the last vector broadcasted by the rest (u'_i for $i \in [1, n]$), and when an update arrives from node p_j then u'_j is set to the new value, and the estimate vector is recalculated. The algorithmic implementation is provided in algorithm 3.3.

ALGORITHM 3.3: The Decentralized Algorithm of the Geometric Approach

INITIALIZATION: AT A NODE p_i :

- Broadcast a message containing the initial statistics vector and update v'_i to hold the initial statistics vector. Upon receipt of messages from all the nodes, calculate the estimate vector ($e(t)$).

PROCESSING STAGE AT A NODE p_i :

- Upon the arrival of new data on the local stream, recalculate $v_i(t)$, and $u_i(t)$, and check if $B(e(t), u_i(t))$ remains monochromatic. If not, broadcast the message $\langle i, v_i(t) \rangle$ and update v'_i to hold $v_i(t)$.
 - Upon receipt of a new message $\langle j, v_j(t) \rangle$, update v'_j to hold $v_j(t)$, recalculate $e(t)$, and check if $B(e(t), u_i(t))$ is monochromatic. If $B(e(t), u_i(t))$ is not monochromatic, broadcast the message $\langle i, v_i(t) \rangle$ and update v'_i to hold $v_i(t)$.
-

Coordinator-based Setting:

In the coordinator-based setting, a coordinator node is designated that assumes responsibility for collecting the local statistics vectors from the nodes, to calculate the estimate vector to distribute it to the rest. This way there are no more expensive broadcast operations, and this is a task completed by the coordinator. In addition to this modification, another element of the coordinator-based architecture is that it employs a mechanism for balancing the local statistic vectors of a subset of nodes. To achieve this, the second equation of eq. (3.2) is slightly modified to look like the following:

$$v_i(t) = e(t) + \Delta u_i(t) + \frac{\delta_i}{w_i} \quad (3.3)$$

with this adjustment, a so-called *slack vector* is additionally sent from the coordinator when it is observed that two nodes p_i and p_j cancel each other out: $\Delta_i(t) = -\Delta_j(t)$. The total sum of the slack vectors sent to the nodes is 0, that way in a global perspective the estimate statistics vector is not influenced, yet the reinstated values of the nodes drift vector lead to better performance.

Unlike the Decentralized algorithm, the coordinator-based counterpart provides a more sophisticated approach that is preferable to be explained swiftly instead of providing a complicated algorithmic implementation.

The coordinator-based algorithm involves the central coordinator managing the local statistics and slack vectors from distributed nodes to monitor a global threshold function efficiently. The process starts with each node sending its initial statistics vector to the coordinator, which then calculates an estimate vector and broadcasts it to all nodes. As new data arrives, nodes locally update their statistics and drift vectors and check if their local constraints are violated. If a violation occurs, the node sends a report to the coordinator. The coordinator attempts to balance the system by adjusting slack vectors among a subset of nodes to maintain the correctness of the

estimate vector. If balancing fails, a new estimate vector is calculated and broadcasted. This way communication overhead is reduced by restricting the interactions between the coordinator and a subset of nodes rather than frequent broadcasts from one node to the rest.

Geometric Property:

To form an understanding of the two presented algorithms and grasp the geometric connection, consider the task of decomposing a global monitoring task into local constraints on data streams. Each node p_i verifies if its local constraint has been violated. If none of these constraints are violated, the query result remains unchanged, requiring no communication.

The global statistics vector $u(t)$ at time t is the weighted average of the drift vectors $v_i(t)$ held by the nodes:

$$u(t) = \frac{\sum_{i=1}^n w_i v_i(t)}{\sum_{i=1}^n w_i} \quad (3.4)$$

This implies that the global statistics vector $u(t)$ is within the convex hull of the drift vectors:

$$u(t) \in \text{Conv}(v_1(t), v_2(t), \dots, v_n(t)) \quad (3.5)$$

To monitor the local constraints, we use Theorem 1, which states that for vectors $x, y_1, y_2, \dots, y_n \in \mathbb{R}^d$, the convex hull $\text{Conv}(x, y_1, y_2, \dots, y_n)$ can be bounded by n d -dimensional balls $B(x, y_i)$ each centered at $\frac{x+y_i}{2}$ with radius $\frac{\|x-y_i\|}{2}$:

$$\text{Conv}(x, y_1, y_2, \dots, y_n) \subseteq \bigcup_{i=1}^n B\left(\frac{x+y_i}{2}, \frac{\|x-y_i\|}{2}\right) \quad (3.6)$$

In our scenario, we construct n balls $B(e(t), v_i(t))$ where each ball is centered at $\frac{e(t)+v_i(t)}{2}$ with radius $\frac{\|e(t)-v_i(t)\|}{2}$, ensuring that:

$$\text{Conv}(e(t), v_1(t), v_2(t), \dots, v_n(t)) \subseteq \bigcup_{i=1}^n B\left(\frac{e(t)+v_i(t)}{2}, \frac{\|e(t)-v_i(t)\|}{2}\right) \quad (3.7)$$

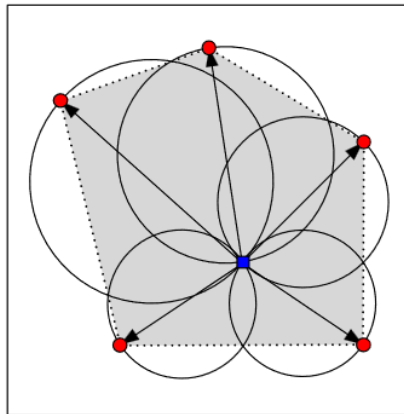


Figure 3.7. Drift vectors comprising the convex hull.[9]

Each node checks if the ball $B(e(t), v_i(t))$ is monochromatic. If a ball is monochromatic red, $\{x \mid f(x) < r\}$, no communication is required. If any ball contains green vectors, $\{y \mid f(y) \geq r\}$, the node reports its local statistics, indicating a local constraint breach. Figure 3.7 illustrates this concept, showing the drift vectors $v_i(t)$ for a 2-dimensional statistic on a plane. The convex hull of

the drift vectors is shown in gray, and the union of the balls $B(e(t), v_i(t))$ encompasses this convex hull, allowing nodes to collectively monitor the global constraint.

In conclusion, the geometric method presented a substantial breakthrough in monitoring threshold functions over distributed data streams. The findings of this work provided a pathway for implementing functional monitoring mechanisms, in distributed systems, crucial for the synchronization of Distributed Learning (DL) processes, and by extension Federated Learning (FL). These mechanisms can substantially reduce the frequency of communication and achieve efficient learning by applying some simple redefinitions to fit our problem's requirements. Specifically, replacing *nodes* with *clients*, *statistics* with *model parameters*, and providing a fitting *monitor function* of the *local statistics* presents a methodology directly applicable to the federated setting at hand.

3.4.2 Functional Dynamic Averaging

The Groundwork:

The encouraging findings of the geometric method[9], propelled further research leading to the publication of "Sketch-based Geometric Monitoring of Distributed Stream Queries"[76] M.Garofalakis, V.Samoladas and D.Keren with the first two proceeding with "Functional Geometric Monitoring for Distributed Streams" [77].

Sketch-based Geometric Monitoring[76] extends the geometric method concept by integrating AMS (Alon-Matias-Szegedy) sketches[78] with geometric monitoring, enabling efficient and scalable monitoring of complex, non-linear aggregate queries. This combination allows for significant dimensionality reduction, providing error bounds and approximation guarantees, and further reducing communication costs, making the method robust against high variability and skew in data streams.

Additionally, the subsequent introduction of Functional Geometric Monitoring (FGM) comprised a conjunction of the fundamental ideas of geometric monitoring[9] (GM) and the sketches based approach[76].

Functional Geometric Monitoring (FGM) employs non-linear functions to project local statistics vectors, or summary vectors, enhancing the flexibility and performance of monitoring even in scenarios with high data variability and skewness. This approach further reduces communication costs by maintaining performance within adaptable and strict bounds, closely approximating the coordinator-based version of GM (see section 3.4.1). FGM also achieves a clear delineation between monitoring logic and distributed system issues, providing a viable solution for the monitoring logic of any general-purpose middleware platform. Additionally, FGM introduces the utilization of sketches to approximate local summary vectors and incorporate them into the geometric monitoring framework, enabling accurate monitoring of complex distributed stream queries.

Methodology:

Building on the promises of previously explored synchronization methods, Functional Dynamic Averaging (FDA) was introduced by V.Konidaris and V.Samoladas [79] as a sophisticated synchronization strategy for coordinator-based distributed architectures. FDA presents a compelling approach that can be modified to fit any Distributed or Federated Learning architecture that requires synchronization as it adopts the characteristic of FGM for separation of the learning process with the undelaying monitoring logic. The application of the aforementioned approach resulted in substantial mitigation of communication in distributed environments accomplishing not only swift training convergence but also high levels of accuracy, ranking higher than comparable synchronization methods developed in the same context.

The proposed approach aims to create a global monitoring task that ensures only significant changes in local models are reported, thereby preserving network bandwidth and reducing latency. Similar to Geometric Monitoring (GM) logic, the objective is to approximate the global monitoring task by a set of local constraints that can be monitored by all clients. A fundamental principle of Distributed Machine Learning (DML) is that the global model maintains a good perception of the obtained knowledge, resulting in better performance when it is closely approximated by the weighted average of all local models. Thus, the proposed constraints must guarantee this initial condition.

The global model is notated as $\bar{w}$ and is generally unknown throughout the learning process. Let $w_t^{(k)}$ be the model of client k at moment t , then the global model is calculated as follows:

$$\bar{w} = \sum_{k=1}^n \frac{n_k}{n} w_t^{(i)} \quad (3.8)$$

This indicates, however, that at any given moment t the clients need to be transmitting their model to the server. Such an implementation would be unreasonable as it would incur particularly frequent communications, deteriorating the performance. Naturally, to address this issue FDA, similar to other synchronization methods (including BSP, TAP, and SSP), works in rounds, and when the end of a specific round is signaled then the perceived global model is updated. In other words, at the end of a learning round, all clients send their local models to the server for aggregation as dictated by eq. (3.8). The newly-calculated global model is redistributed to the clients providing an estimate of the global model.

Nonetheless, the question that needs to be answered is how the end of a round is determined. The method of FDA applies a principle that stems from the FGM approach, utilizing a non-linear projection function to transform the local statistics-summary vectors into real numbers, which are collectively monitored. In FDA the role of the projection function is assumed by the variance of the local model in comparison with the global estimate. The reason why variance, is selected is to measure how accurate is the perception we maintain for the global model. In the case that local models start deviating substantially from the global estimate, this means that a global update is required to bridge this gap, or else the global model's accuracy will deteriorate.

The reason the above is important to answer the question posed is because, in order to identify the end of a round, the value of the variance needs to surpass a thresholded value Θ . Θ consists a hyperparameter of FDA that is defined at the start of a training ground and can be subsequently reconfigured depending on the needs of the synchronization algorithm. The mathematical representation of the above follows:

The global model is notated as $\bar{w}$ and is generally unknown throughout the learning process. Let $w_t^{(k)}$ be the model of client k at moment t , then the global model is calculated as follows:

$$\frac{1}{n} \sum_{k=1}^n \|w_t^{(k)} - \bar{w}_t\|^2 \leq \Theta \quad (3.9)$$

The condition depicted above is the most crucial part of FDA and is named Round Terminating Condition (RTC) with the left part presenting model variance and the right the threshold Θ . As long as RTC is not violated it can be induced that the global estimate of the model is accurate and performs efficiently. That said, FDA introduced three major approaches -Naive FDA, Linear FDA, and Sketch FDA- differing mainly in how they monitor the Round Termination Condition (RTC) and approximate the variance of the local models. Each method has a different approach to balancing the trade-off between communication overhead and the accuracy of synchronization.

RTC Monitoring:

As stated previously in section 3.4.2, The underlying technique of RTC monitored is entailed by the fundamental principles of FGM[77]. In this context, Assume there are n clients in the learning network. Each of these clients has a local stream generated locally or collected. Let $S_k(t) \in \mathbb{R}^m$ denote the *local state vector* of client k in moment t . The *global state vector* can be denoted as $S(t) \in \mathbb{R}^m$ and is computed as the average of all local states.

$$S(t) = \frac{1}{k} \sum_{i=1}^k S_i(t) \quad (3.10)$$

The goal is to monitor a bounded condition on the global model, by projecting the global function into real numbers and comparing the projection with threshold Θ . The projection function is non-linear and is denoted as $F : \mathbb{R}^m \rightarrow \mathbb{R}$ leading to the following condition:

$$F(S(t)) \leq \Theta \xRightarrow{3.10} F\left(\frac{1}{k} \sum_{i=1}^k S_i(t)\right) \leq \Theta \quad (3.11)$$

Additionally, let t_0 be the time when the latest communication round started. At this initial time, the local models are synchronized, so $w_{t_0}^{(1)} = w_{t_0}^{(2)} = \dots = w_{t_0}^{(n)} = \bar{w}_{t_0}$. The *update of learner k* at step t is the difference between the weights at step t and the weights at the last synchronization t_0 which represents the estimation of the global state. Thus the average update can also be induced as the mean of the clients' updates. Overall:

$$\begin{cases} \Delta_t^{(k)} = w_t^{(k)} - w_{t_0}^{(k)}: \text{Update of learner } k \text{ at step } t \\ \bar{\Delta}_t = \frac{1}{n} \sum_{k=1}^n \Delta_t^{(k)}: \text{Average update} \end{cases} \quad (3.12)$$

With these definitions in mind we can proceed and reformulate the norm calculation on the left side of RTC eq. (3.9):

$$\begin{aligned} \frac{1}{n} \sum_{k=1}^n \|w_t^{(k)} - \bar{w}_t\|^2 &= \frac{1}{n} \sum_{k=1}^n \|(w_t^{(k)} - w_{t_0}^{(k)}) - (\bar{w}_t - w_{t_0}^{(k)})\|^2 \\ &= \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)} - \bar{\Delta}_t\|^2 \\ &= \frac{1}{n} \sum_{k=1}^n (\|\Delta_t^{(k)}\|^2 - 2\Delta_t^{(k)} \cdot \bar{\Delta}_t + \|\bar{\Delta}_t\|^2) \\ &= \left(\frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 \right) - 2 \left(\frac{1}{n} \sum_{k=1}^n \Delta_t^{(k)} \right) \cdot \bar{\Delta}_t + \|\bar{\Delta}_t\|^2 \\ &= \left(\frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 \right) - \|\bar{\Delta}_t\|^2 \end{aligned} \quad (3.13)$$

Thus, conceptually we can set the local state of client k and the function F in the following manner to satisfy our objectiveeq. (3.11):

$$\begin{cases} S_k(t) = [\|\Delta_t^{(k)}\|^2, \Delta_t^{(k)}]^T \in \mathbb{R}^{m+1}, \\ F([v, x]^T) = v - \|x\|^2 \end{cases} \quad (3.14)$$

Using these aforementioned definitions, the RTC can be expressed as:

$$F(S(t)) \leq \Theta,$$

Practical Considerations

Defining the local state $S_k(t)$ in the described manner in eq. (3.14) can be resource-intensive for the network, as the local update $\Delta_t^{(k)}$ has m dimensions, corresponding to the number of weights in the local client's model. Therefore, our goal is to examine different ways to define the local states and the function F to minimize network overhead. This process leads to the presentation of the three proposed FDA approaches, each opting for a different approach of dimensionality reduction on the local updates $\Delta^{(i)}$. The idea is that as the approximation of the actual RTC becomes stricter, the cost of communication increases. This trade-off is inevitable and will be studied further, with the three methods being presented with increasing communication cost, yet decreasing approximation accuracy.

Nonetheless, at this point, the general algorithmic approach of FDA is attached to maintain a common offset before the upcoming branching in the three approaches. The following algorithm is a product of M. Theologitis thesis-dissertation[80].

ALGORITHM 3.4: FDA: FL training with Approximate RTC Monitoring

Require: $f(w)$: Stochastic objective function with parameters $w \in \mathbb{R}^d$
Require: K : The number of clients indexed by k
Require: Θ : Monitoring threshold
Require: $S_k(t)$: The local state for client k where $S_k(t) \in \mathbb{R}^p$ and $p \ll d$
Require: $F(x)$: A function $F : \mathbb{R}^p \rightarrow \mathbb{R}$ such that $F(S(t)) \leq \Theta$ implies the RTC
Require: η : Step size (learning rate)
Require: s : The number of local steps
Require: b : The local mini-batch size
Server executes:
 initialize w_1
for each round $t = 1, 2, \dots$ **do**
 Broadcast w_t to all clients
 repeat
 for each client $k = 1, \dots, K$ **in parallel do**
 $S_k(t) \leftarrow \text{ClientTrain}(k)$
 end for
 until $F(S(t)) > \Theta$
 for each client $k = 1, \dots, K$ **in parallel do**
 $w_{t+1}^{(k)} \leftarrow$ (download the model of client k)
 end for
 $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^{(k)}$
end for
ClientTrain(k):
 $\mathcal{B} \leftarrow$ (Choose s batches of size b from $\mathcal{P}_k$)
 for each batch $B \in \mathcal{B}$ **do**
 $w \leftarrow w - \eta \nabla f_B(w)$
 end for
 return $S_k(t)$ to server

Naive Approximation

The Naive FDA unlike Naive, which ignores the local state vector, simplifies the problem by reducing the local state to a single scalar value, specifically the squared norm of the local update vector. This approach is straightforward and incurs minimal computation overhead, making it suitable for scenarios where communication costs are critical. The local state for client k at time t , along with the projection function F are reformed:

$$\begin{cases} S_k(t) = \|\Delta_t^{(k)}\|_2^2 \\ F(v) = v \end{cases} \quad (3.15)$$

The function $F(v) = v$ is used to aggregate the states across clients. The RTC is checked using the condition $F(S(t)) \leq \Theta$. The proof that Naive FDA satisfies the RTC involves starting from the condition we want to prove and with logical equalities conclude in a known condition (RTC):

$$\begin{aligned} F(S(t)) \leq \Theta &\iff \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 \leq \Theta \\ &\iff \frac{\|\bar{\Delta}_t\|^2 \geq 0}{\iff} \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - \|\bar{\Delta}_t\|^2 \leq \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 \leq \Theta \\ &\stackrel{3.13}{\iff} \frac{1}{n} \sum_{k=1}^n \|\mathbf{w}_t^{(k)} - \bar{\mathbf{w}}_t\|^2 \leq \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 \leq \Theta \\ &\implies \frac{1}{n} \sum_{k=1}^n \|\mathbf{w}_t^{(k)} - \bar{\mathbf{w}}_t\|^2 \leq \Theta \quad \underline{\text{RTC}} \quad (\text{Equation (3.9)}) \end{aligned} \quad (3.16)$$

Since this aggregation overestimates the actual variance, ensuring $F(S(t)) \leq \Theta$ implies that the RTC is met. This approach is the most simple requiring merely a 1-dimensional scalar value to be communicated to monitor RTC. In most cases, accuracy is not heavily impacted, yet this approximation is based on a loose condition, meaning that deviation from the required results can be noticed in some cases.

3.4.3 Linear Approximation

The Linear FDA method provides a more refined approach by reducing the local update vector $\Delta_t^{(k)}$ to a scalar product with a unit vector ξ , meaning $\xi \cdot \Delta_t^{(k)} \in \mathbb{R}$. This method balances computational efficiency and accuracy in estimating the variance of the updates. The local state for client k at time t , along with the projection function F , are defined as:

$$\begin{cases} S_k(t) = \left[\|\Delta_t^{(k)}\|_2^2, \langle \xi, \Delta_t^{(k)} \rangle \right]^\top \\ F(v, x) = v - x^2 \end{cases} \quad (3.17)$$

The function $F(v, x) = v - x^2$ is used to aggregate the states across clients. The RTC is checked using the condition $F(S(t)) \leq \Theta$. The proof that Linear FDA satisfies the RTC follows the same methodology as in Naive:

$$\begin{aligned} F(S(t)) \leq \Theta &\iff \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - \frac{1}{n} \sum_{k=1}^n (\xi \cdot \Delta_t^{(k)})^2 \leq \Theta \\ &\stackrel{\xi^2=1}{\iff} \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - \|\bar{\Delta}_t\|^2 \leq \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - (\xi \cdot \bar{\Delta}_t)^2 \leq \Theta \\ &\stackrel{3.13}{\iff} \frac{1}{n} \sum_{k=1}^n \|\mathbf{w}_t^{(k)} - \bar{\mathbf{w}}_t\|^2 \leq \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - (\xi \cdot \bar{\Delta}_t)^2 \leq \Theta \\ &\implies \frac{1}{n} \sum_{k=1}^n \|\mathbf{w}_t^{(k)} - \bar{\mathbf{w}}_t\|^2 \leq \Theta \quad \text{RTC (Equation (3.9))} \end{aligned} \quad (3.18)$$

This method ensures that the variance is accurately estimated, and the condition $F(S(t)) \leq \Theta$

implies the RTC is met. This approach balances communication efficiency and accuracy, making it suitable for scenarios requiring a more precise estimate than the Naive method. A key consideration, nonetheless for this approach is the selection of ξ . A random selection of ξ is likely to perform inadequately (leading to early termination of a round), as it may be nearly orthogonal to Δ . A preferable choice would be a vector that is correlated with Δ . One heuristic option is to use Δ_0 (normalized to have a unit norm), which is the update vector from right before the current round commenced. Each node can estimate this independently without needing communication, as it is the difference between the last two models, $\tilde{w}_0 - \tilde{w}_1$, sent by the coordinator.

3.4.4 Sketch FDA

The Sketch FDA method utilizes AMS sketches[76] to compress local update vectors, achieving an optimal balance between communication efficiency and accuracy. This technique is particularly advantageous in scenarios involving high-dimensional data that require significant dimensionality reduction. The underlying logic and analysis presented here follow the methodology outlined by Frangias[81] in his comprehensive study on FDA.

The AMS sketch denoted as $\text{sk}(\mathbf{v})$, is a technique that compresses a high-dimensional vector into a significantly smaller matrix. For a vector $\mathbf{v} \in \mathbb{R}^M$, the sketch $\text{sk}(\mathbf{v})$ is defined as:

$$\text{sk}(\mathbf{v}) = \Xi = [\xi_1 \quad \xi_2 \quad \dots \quad \xi_d], \quad (3.19)$$

where each ξ_i is a sub-vector of dimension d , and typically $d \cdot m \ll M$.

The sketch function is linear and can be computed in $O(dM)$ steps. Furthermore, there exists a function, denoted as $\mathcal{M}_2$, that accurately estimates the squared norm of a vector $\mathbf{v}$ using its sketch $\text{sk}(\mathbf{v})$. This function is defined as:

$$\mathcal{M}_2(\text{sk}(\mathbf{v})) = \text{median}_{i=1, \dots, d} \|\xi_i\|^2. \quad (3.20)$$

For $m = O\left(\frac{1}{\epsilon^2}\right)$ and $d = O\left(\log \frac{1}{\delta}\right)$, it is proven that with high probability $(1 - \delta)$, the squared norm $\|\mathbf{v}\|^2$ can be approximated within a factor of $(1 - \epsilon)$ to $(1 + \epsilon)$:

$$(1 - \epsilon)\|\mathbf{v}\|^2 \leq \mathcal{M}_2(\text{sk}(\mathbf{v})) \leq (1 + \epsilon)\|\mathbf{v}\|^2. \quad (3.21)$$

In our context, the large vector requiring compression is the local update $\Delta_t^{(k)} \in \mathbb{R}^M$. Consequently, the local state $S_k(t)$ and the function F for the sketch method are defined as follows:

$$\begin{cases} S_k(t) = \begin{bmatrix} \|\Delta_t^{(k)}\|^2 & \text{sk}(\Delta_t^{(k)}) \end{bmatrix}^T \in \mathbb{R}^{1+d \times m}, \\ F \begin{pmatrix} v \\ \Xi \end{pmatrix} = v - \frac{1}{1+\epsilon} \mathcal{M}_2(\Xi). \end{cases} \quad (3.22)$$

To establish that the RTC holds, we proceed as follows:

$$\begin{aligned}
F(S(t)) \leq \Theta &\iff \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - \frac{1}{1+\epsilon} \cdot \frac{1}{n} \sum_{k=1}^n \mathcal{M}_2(\text{sk}(\Delta_t^{(k)})) \leq \Theta \\
&\stackrel{\text{linearity}}{\iff} \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - \frac{1}{1+\epsilon} \cdot \mathcal{M}_2(\text{sk}(\Delta_t)) \leq \Theta \\
&\stackrel{3.21}{\iff} \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - \|\bar{\Delta}_t\|^2 \leq \frac{1}{n} \sum_{k=1}^n \|\Delta_t^{(k)}\|^2 - \frac{1}{1+\epsilon} \mathcal{M}_2(\text{sk}(\Delta_t)) \leq \Theta \\
&\stackrel{3.13}{\implies} \frac{1}{n} \sum_{k=1}^n \|\mathbf{w}_t^{(k)} - \bar{\mathbf{w}}_t\|^2 \leq \Theta \quad \underline{\text{RTC}} \text{ (Equation (3.9))} \tag{3.23}
\end{aligned}$$

3.4.5 Multiprogramming & Synchronization Primitives in DML

Learning in Distributed Machine Learning Systems is a complicated process that involves one or more servers that orchestrate and manage a great number of worker nodes. Especially if we consider the applicability of FL in the Internet of Things (IoT) scenarios, the number of coordinated nodes can be humongous. For this reason, a plethora of synchronization mechanisms need to be employed on the server side to ensure that updates to the model parameters are consistent and free from conflicts. Multiprogramming and synchronization primitives play a crucial role in achieving this goal by managing access to shared resources and preventing race conditions. The purpose of this section is to highlight the principal mechanisms utilized in this context providing a comprehensive overview.

Locks are fundamental in distributed machine learning for protecting shared resources such as model parameters, ensuring that only one node can modify these resources at a time. Locks help prevent data corruption by enforcing mutual exclusion. However, they can lead to high contention and potential deadlocks. Techniques like lock striping and sharding are used to reduce contention by breaking down large locks into smaller, more manageable parts, allowing greater parallelism and reducing the likelihood of performance bottlenecks

Mutexes (mutual exclusions) provide a robust mechanism for ensuring exclusive access to critical sections of code, particularly when updating shared model parameters. They prevent multiple nodes from making concurrent updates that could lead to inconsistent states. Challenges include avoiding priority inversion, where a high-priority task is blocked by a lower-priority one. Priority inheritance protocols are employed to temporarily elevate the priority of the task holding the mutex, mitigating this issue.

Semaphores manage access to limited resources such as GPUs in distributed machine learning. By controlling the number of concurrent accesses, they ensure efficient and fair resource utilization. Semaphores prevent resource exhaustion with timeout mechanisms that allow tasks to fail gracefully if they cannot acquire a semaphore within a specified time. Proper exception handling ensures semaphores are released correctly, preventing resource leaks and maintaining system stability.

Condition Variables synchronize nodes based on specific conditions or events, allowing threads to wait until certain conditions are met. This is crucial in distributed machine learning for coordinating updates and ensuring that nodes do not proceed until necessary conditions are satisfied. Condition variables must handle challenges like missed signals and spurious wakeups. Robust

signaling mechanisms, combined with atomic variables, ensure that condition checks are reliable and that threads are correctly awakened only when the desired conditions are truly met.

Read-Write Locks: Read-write locks are particularly useful in scenarios where model parameters are read frequently but updated infrequently. They allow multiple nodes to read data simultaneously while ensuring exclusive access for write operations. This improves efficiency and maintains consistency in model states. Read-write locks must address writer starvation, which occurs when writers are perpetually blocked by continuous read operations. Fairness policies that prioritize writers after a certain number of reads help mitigate this issue, ensuring balanced access to resources.

Ensuring Model Consistency:

Ensuring model consistency in distributed machine learning involves carefully designing synchronization mechanisms to prevent data corruption and ensure coherent updates. In this context, multiprocessing and synchronization primitives are essential for maintaining model consistency and ensuring efficient resource utilization. By leveraging locks, mutexes, semaphores, condition variables, and read-write locks, server-side systems can synchronize updates, prevent race conditions, and manage access to shared resources effectively. Understanding and correctly implementing these primitives is crucial for building robust and high-performance distributed machine learning systems. Key considerations to achieve the above include:

1. *Avoiding Deadlocks:* In a distributed setting, deadlocks can occur when multiple nodes wait indefinitely for resources held by each other. Techniques such as establishing a strict order for acquiring locks and implementing timeout mechanisms can help avoid deadlocks.
2. *Minimizing Lock Contention:* Lock contention arises when multiple nodes attempt to acquire the same lock, leading to delays. Reducing the granularity of locks and minimizing the duration of critical sections can help alleviate lock contention.
3. *Preventing Priority Inversion:* Priority inversion occurs when a high-priority node is blocked by a lower-priority node holding a lock. Implementing priority inheritance protocols can mitigate this issue by temporarily elevating the priority of the lower-priority node.
4. *Efficient Barrier Synchronization:* Barrier synchronization ensures that all nodes reach a certain point in computation before proceeding. This is critical in distributed training phases, ensuring that all nodes have completed their updates before the next iteration begins.

3.5 NS3-FL: Bridging Data & Network Management

It has become clear to this point that one of the key inspirations of this research study is to introduce a user-friendly interface for federating any machine learning pipeline in a unified approach that considers diverse datasets, algorithmic implementations and network interaction. Despite the great work being conducted in the field of federated learning by countless researchers, the above venture is something that has yet to be prioritized by an industry leader to produce.

Nonetheless, the paper *ns3-fl: Simulating Federated Learning with ns-3* by Ekaireb et al.[82] in 2022, introduced an innovative approach in which studied the amalgamation of a PyTorch-based FL simulator (flsim) with a network simulator (ns-3) to produce a unified simulation framework. In this section, we will discuss the architecture and methodology of the proposed framework, that consisted the groundwork for the conjunction of our federated learning setup with a network simulator.

3.5.1 Framework Overview

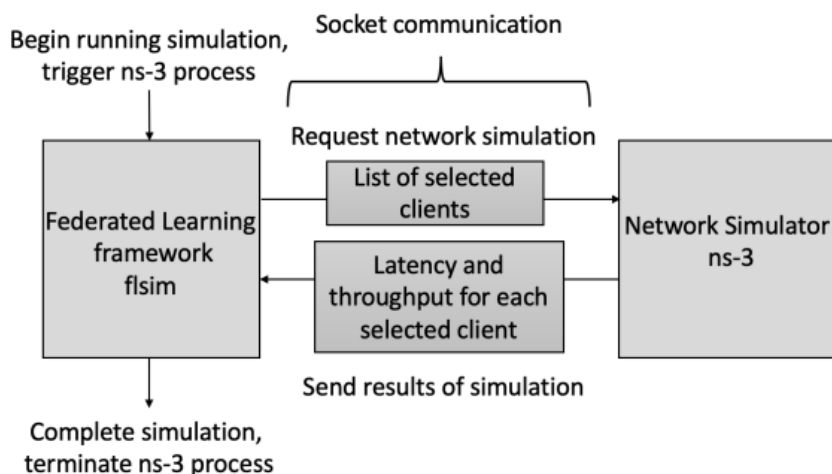


Figure 3.8. Architectural overview of ns3-fl.

Figure 3.8 depicts the top-level architecture of ns3-fl, that is decomposed in two software blocks in accordance with the previous description: The FL framework of flsim for data distribution and FL processing, and the Ns3 Network simulator for network management and monitoring.

The depicted functionality can be summarized as follows: At the start of each FL training round, flsim initiates a network simulation request to ns-3, specifying the selected clients. Ns-3 performs the network simulation and returns latency and throughput data for each client. Flsim, in turn, utilizes these network statistics to calculate convergence time and average throughput for the training round.

3.5.2 Learning, Network, and Power Models

Learning Model

The learning model in ns3-fl supports both synchronous and asynchronous FL algorithms. The canonical setting is a star-topology network (see section 2.6.1) with N client nodes connected to a central server. The learning process, represent a typical scenario of FL where each client trains on its local dataset and sends model updates to the server. The objective is to minimize the overall loss function aggregated from all clients' local losses. All the functions in regard with the learning process are processed by the flsim block.

Network Model

The network model in ns3-fl uses ns-3 to simulate realistic network conditions. Key metrics such as latency and throughput are considered for each client. These metrics influence the overall convergence time and performance of the FL process. The framework supports various network configurations, including wireless and ethernet connections.

Power Model

The power model calculates the energy consumption of FL training on client devices by modeling the consumption for two types of appliances: Raspberry Pi (RPi) 4 and 400. In addition, to energy consumption, the time required for training for a specific device is calculated. as the

two elements are interconnected $E = P \cdot t$. For the computations the model takes into account machine-learning specific parameters such as the number multiply-accumulate (MAC) operations, along with coefficient parameters calculated by applying linear regression on the power measurements for the two devices. This model helps in understanding the performance; including energy efficiency and time for training of different FL configurations.

3.5.3 Implementation

The implementation of ns3-fl involves creating Client and Server applications within ns-3, to mimic the data transmission conducted for the global model distribution and the model updates. The communication between flsim and ns-3 is managed through the interprocess communication protocol that is based on *TCP Sockets*. The framework can be easily extended to include new data, algorithm, and network models, as it is highly configurable through configuration files. Figure 3.9 presents an example encompassing the workflow during a complete training process. The communication between ns-3 and flsim is facilitated by four primary socket commands:

#	Command	Use
0	RESPONSE	Send results from ns-3 simulation
1	STARTSIM	Schedule a round simulation in ns-3
2	EXIT	Terminate ns-3 process
3	ENDSIM	For async, alerts that ns-3 round ended

Table 3.2. Summary of socket commands used in ns3-fl.

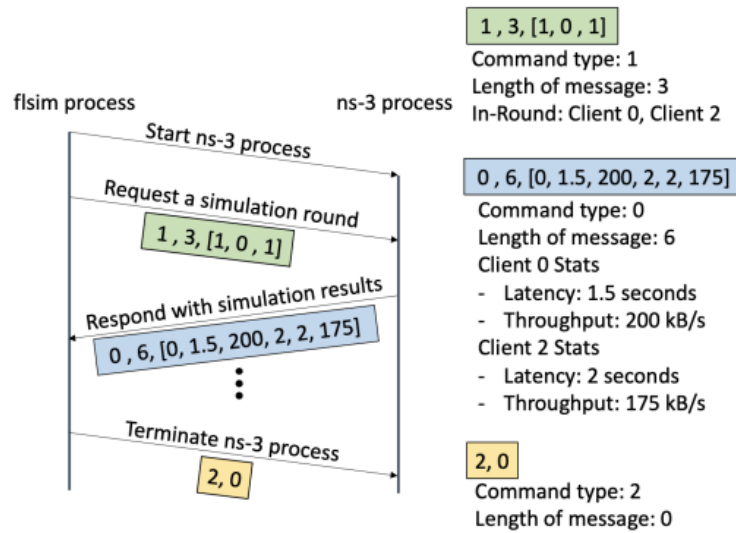


Figure 3.9. Synchronous federated learning training workflow.

Observing the flow diagram the FL simulator (flsim) after starting, initializes the network simulator (NS3). Subsequently, to start a simulation for a training round a request is sent of *commandtype* : 1 including the list of participating clients. Then NS3 runs a network simulation round using the selected set of participants. There is a mapping between the two simulators in order to match their corresponding clients. When the round is complete, a response is sent back to flsim containing the simulated latencies and throughput of the participants. That way the interprocess communication is finalized for this communication round. This process is clearly depicted in fig. 3.9 along with an explained example.

3.5.4 Simulation Results

The ns3-fl framework was validated through both a real deployment of Raspberry Pis and large-scale simulations. The validation focused on measuring the framework’s accuracy, training time, and energy consumption, comparing these metrics against those obtained from the real deployment. The results demonstrated that ns3-fl accurately replicates real-world federated learning scenarios, with errors not exceeding 1.5%. An illustration of the results is provided in table 3.3. For robustness, the analysis also included varying datasets, local epochs, and client selection strategies.

Experiment	MNIST	FashionMNIST	CIFAR-10
Real, RPi 400	18.32 ± 2.12	29.69 ± 1.10	33.86 ± 6.57
ns3-fl, RPi 400	$20.55 \pm 1.1E-5$	$27.78 \pm 1.9E-5$	$30.44 \pm 1.4E-5$
Real, RPi 4	18.91 ± 0.84	27.85 ± 1.17	30.30 ± 1.47
ns3-fl, RPi 4	$19.01 \pm 1.1E-5$	$27.44 \pm 1.9E-5$	$30.15 \pm 1.4E-5$

Table 3.3. Comparison of computational time (in seconds) between real deployment and ns3-fl simulations on different datasets.

Additionally, a series of simulations was conducted under different data distributions and network conditions to assess the performance and scalability of various federated learning configurations. These simulations provided valuable insights into how different setups affect the overall efficiency and scalability of federated learning systems.

3.5.5 Conclusion

The ns3-fl framework provides a compelling tool for simulating federated learning under realistic network conditions. By integrating flsim with ns-3, researchers can validate their FL algorithms considering the interplay between data, algorithm, and network. The framework’s ability to accurately replicate real-world scenarios makes it a valuable resource for advancing federated learning research, as well as, a great starting base for implementing novel approaches in unifying network and data management, similar to the one described in our thesis.

Part

Technical Implementation

Chapter 4

Environment Replication

In this section, we will provide a comprehensive overview of the tools and frameworks utilized for the project's implementation. The integration of various platforms and communication protocols is critical to replicating the environment in which this research was conducted. This includes frameworks for deep and federated learning, network simulation tools, and interprocess communication mechanisms.

To be precise, *TensorFlow* and the *Keras* library were used to develop and train deep learning models, which were then incorporated into the *Flower framework* for federated learning. Additionally, *Ns-3* simulated the network environment to provide realistic conditions for the federated learning process. Mechanisms for interprocess communication were essential. In this regard, *POSIX sockets* enabled communication between the network simulation (*Ns-3*) and the federated learning framework (*Flower*), ensuring network conditions could dynamically influence the federated learning results while *gRPC* facilitated efficient communication between the flower central server and clients, orchestrating the learning process across multiple devices. A brief introduction of the aforementioned tools follows.

4.1 Tensorflow & Keras Deep Learning Platforms

4.1.1 TensorFlow Framework

TensorFlow, developed by Google, is an open-source library for numerical computation and large-scale machine learning (ML). Initially designed to extend ML applicability beyond Google's computational infrastructure, TensorFlow caters to the needs of both inexperienced practitioners and expert researchers. It supports tasks such as deep learning, neural networks, and general numerical computations on CPUs, GPUs, and TPUs. TensorFlow's strength lies in its extensive community, robust scalability, and versatility, enabling it to train and run deep neural networks for tasks like image recognition and natural language processing. It also provides robust tools for model training and optimization, along with infrastructures for efficient dataset handling.

The framework uses a flexible architecture that lets users deploy computation across a variety of platforms (e.g., CPUs, GPUs, and TPUs), including mobile and edge devices. TensorFlow's computational model uses dataflow graphs, where nodes represent mathematical operations and edges represent data arrays (tensors) that flow between them. It also has extended its support in distributed setups with TensorFlow Distribute API that provides the capability of orchestrating multidevice distributed learning.

Tensors data arrays are the fundamental building blocks in TensorFlow. They are the generalized version of scalars (0D), vectors (1D), and matrices (2D) and can extend to higher dimensions. Tensors facilitate the representation and manipulation of data in machine learning models, allowing for efficient computation across different hardware platforms, including CPUs, GPUs, and

TPUs. Operations on tensors are defined in computational graphs, enabling parallel processing and distributed computing essential for large-scale machine learning tasks.

While Python is the primary language for TensorFlow's front-end API, it also supports several other languages, including C++ and Java, facilitating a broad range of applications. TensorFlow's high-level APIs, like Keras, simplify model building and training, while its low-level APIs offer more flexibility for research and experimentation.

Since its release in 2015, TensorFlow has become one of the most popular frameworks in the AI community, used by major companies like Google, Airbnb, Coca-Cola, and eBay. Its ecosystem includes tools like TensorBoard for visualization and TensorFlow Lite for mobile and embedded devices, making it a comprehensive solution for developing and deploying machine learning models.

4.1.2 Keras Library

Keras is a high-level, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. The Framework's is characterized by great adaptability as it can not only be implemented on top of the majority of deep learning frameworks including TensorFlow, pyTorch and Theano, but also used as a tool to export implementation of model's across different platforms. Keras key characteristics can be summarized as follows:

- ◆ *User-Friendly*: Simplifies the process of building and training models, with the standardization of trivial aspects of the process. This way the user's focus shifts towards improved model implementation without distractions. It also contains built-in models for testing and comparisons.
- ◆ *Modular and Extensible*: Models are composed of layers, that are easily customizable to fit the user's on-paper requirements without requiring technical know-how.
- ◆ *Flexibility*: Provides optimizers, loss functions, and metrics, along with essential functions like fit for training and evaluate for accuracy assessment. For all the above there are ready-to-use implementation while customization is always available for individual purpose needs.
- ◆ *Performance and Scalability*: Widely adopted in industry and research due to its robust performance.

The neural network implementations of keras are based on layer architecture. Layers are the fundamental building blocks used to create models. Each layer performs specific operations on the input data and passes the output to the next layer. Any type of layer located in neural networks in theory can be replicated in Keras including Fully-Connected, Convolutional Recurrent, Pooling or Dropout layers. For instance a fully-connected layer can be employed as a "*Dense layer*" in a Keras model implementation. That way user's can build custom models stacking layers sequentially or using the functional API for more complex architectures.

4.2 Flower Framework: An Efficient Approach to FL

Flower is another open-source framework, supported by a very active community, designed for federated learning orchestration. It facilitates the development, experimentation, and deployment of federated learning systems, enabling machine learning models to be trained across multiple devices or locations while providing efficient communication and privacy handling. In essence, flower provides a comprehensive API to simplify the numerous complications of Distributed learning system management by enabling easy transition from ML to FL setups. This is a vital element

of Flower's functionality as it not only simplifies the process but also ensures that it is ML framework agnostic being able to integrate a wide range of ML platforms. This flexibility is achieved through high-level abstraction, allowing users to seamlessly incorporate their learning processes into the client's infrastructure.

Flower comprises the primary tool for our project's infrastructure offering several advantages for our experimentations. First, it provides the infrastructure both for real-life FL system deployments and simulation settings. This way any given FL system can be tested in optimized simulation setups on a single machine, before proceeding in large-scale real-life deployments. Additionally, with various pre-built aggregation strategies, it simplifies the implementation of federated learning models while providing great customizability, letting users tailor solution to specific implementation needs. Finally the framework is designed for scalability, making it suitable for large-scale deployments, and it accommodates compute heterogeneity, enabling it to operate efficiently across varying devices and environments.

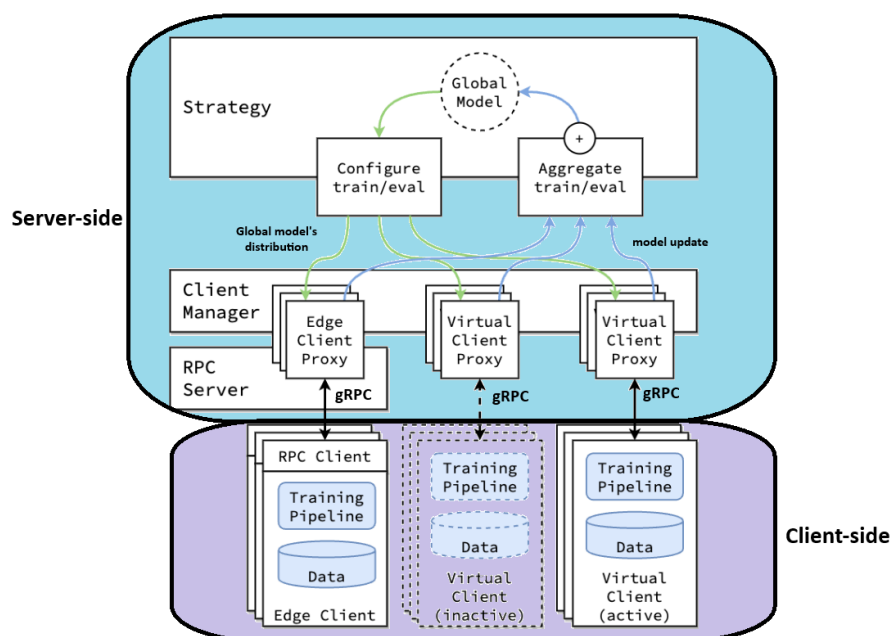


Figure 4.1. Flower architecture with a server, two edge clients and one virtual client.

Source

Based on fig. 4.1, Flower's architecture can be decomposed in three major parts: the server-side, the client-side, and the interprocess communication protocol.

The server-side contains the *global model*, the *strategy*, the *client-manager* that orchestrates the *client-proxies* and an *RPC server*. Strating from the *global model*, it is computed by aggregating the local updates and then redistributed to client following the FL process. The management of the overall federated learning process, including configuring training/evaluation tasks and aggregating results from clients to update the global model is assigned to the *strategy*, that can be a pre-built implementation of a custom logic implemented by the user. The *client proxiestext*, represent an instance of the clients in the server side and act as an intermediary between the RPC server and the edge clients, facilitating communication and data exchange with gRPC call. The *client-manager* assumes the responsibility of coordinating the interactions between the server and cients. Finally the *RPC server* handles remote procedure calls, enabling communication between

the central server and distributed clients.

The client-side of the Flower framework consists of numerous clients, which can be either *virtual* instances or *actual edge devices*. Each client possesses a *local dataset* used for the local training process. In any given training round r , a client may be either *active* or *inactive*. Active clients, whether virtual or edge devices, participate in the training process using their local training pipeline, which can be implemented on their preferred platform. Upon completing training, active clients communicate their results through the *RPC client interface*. Inactive clients remain dormant until they receive a gRPC call from the server, requesting their participation in the subsequent round.

Finally the interprocess communication is conducted with the employment of the *gRPC protocol*. In a brief summary, the protocol requires an RPC server on the server-side and an RPC client on the client side. The RPC Server sends requests and receives responses from the RPC Client on the client-side. Active clients use the gRPC protocol to send their local training results back to the server, ensuring efficient and reliable data transmission. Inactive clients remain dormant until they receive a gRPC call from the server, prompting their participation in the next training round. This mechanism supports seamless and scalable federated learning.

4.3 NS3: An Event-Driven Network Simulator

NS-3 is a discrete-event network simulator primarily used for research and educational purposes. What that means is that the simulator models the operation of a network as a sequence of events occurring at specific points in time. Each event represents a distinct change in the state of the system, such as the arrival of a packet, the completion of a transmission, or the movement of a node. The simulator processes these events in chronological order, updating the state of the network accordingly. This approach allows for detailed and accurate simulation of network behavior over time.

Being open-source and highly extensible, NS-3 provides a realistic simulation environment for modeling and analyzing the performance of networking protocols and architectures. It supports a wide range of communication topologies, both wired and wireless, with detailed implementations that manage various network layers to closely approximate real-life setups. Notably, in wireless topologies, NS-3 can simulate node mobility, interference, and signal propagation.

Additionally, NS-3 offers traffic patterns, data flows, and traffic generators to experiment with different scenarios. This capability allows users to create detailed and accurate network architectures, from which they can extract significant performance metrics such as latency, throughput, and jitter. These features make NS-3 a powerful tool for investigating and simulating various networking scenarios, providing valuable insights into network performance and behavior.

NS-3 benefits from an active community of researchers and developers who continuously strive to enhance the accuracy of its models and the overall functionality of the simulator. It offers comprehensive documentation, tutorials, and example scripts to assist users in getting started. The simulator is designed to be user-friendly, featuring a modular architecture that facilitates easy integration and customization. Moreover, NS-3 supports interoperability with real-world systems and other simulators, thereby enhancing its utility for diverse networking research and development projects. These attributes have established NS-3 as a leading paradigm in the field, extensively used and taught in universities to foster research and education.

4.4 Posix Sockets & gRPC

4.4.1 Posix Sockets

Posix Sockets are fundamental to network communication, providing a mechanism for two machines to exchange data over a network. Essentially, sockets offer a well-established API for inter-process communication in internet and Unix Domain applications. They present the abstraction of a local endpoint in the communication path, represented as file descriptors. Sockets use the TCP/IP protocol suite for communication and support both connection-oriented (using TCP) and connectionless communication setups (using UDP). The robust and reliable mechanisms of POSIX sockets make them a viable choice for federated learning (FL) inter-process communication. One major property of sockets, making them suitable for FL setups, is their ability to handle both blocking and non-blocking communication enabling them to support synchronous and asynchronous FL. In a blocking operation, the program halts until the entire message is sent or received, which can cause potential delays. Conversely, in a non-blocking operation, the program sends or retrieves only the data that is immediately available, preventing it from stalling on slow connections and avoiding many deadlock situations.

In our case, socket communication is restricted to operations within the NS-3 simulator, specifically handling the internal processes and simulations. This includes managing the simulated network events and data exchanges. Once the simulation tasks are completed, sockets are used to communicate the results from NS-3 to the Flower framework. This ensures that the network simulation outcomes are efficiently transferred for further processing in the federated learning workflow managed by Flower, enabling seamless integration and coordination between network simulations and machine learning tasks.

4.4.2 gRPC

gRPC is an open-source RPC (Remote Procedure Calls) framework developed by Google, designed for efficient, low-latency communication between distributed systems. It allows a client to call methods on a server as if they were local, using Protocol Buffers (protobufs) for serializing structured data, ensuring fast and compact communication. gRPC supports multiple programming languages and offers features such as authentication, load balancing, and bidirectional streaming, making it ideal for microservices architectures where seamless and scalable service communication is essential. Its high performance and ease of integration make gRPC a popular choice for modern distributed computing environments.

The process of RPC communication with gRPC begins by defining a service in a `.proto` file using Protocol Buffers syntax, where service methods and message types are specified. This definition is compiled using the protoc compiler to generate both client and server code. The server implements the service by providing the logic for handling RPCs, processing requests, and sending responses. Meanwhile, the client creates a stub from the generated code, enabling it to invoke remote methods on the server as if they were local functions.

It is important to clarify that in this context, "server" and "client" are used in a general sense for communication purposes, not specifically for federated learning structures. The reason for this comment will be better demonstrated with the following example.

In our project the server orchestrates training and evaluation, maintaining client stubs within the client proxies. This setup allows the server to remotely call the clients' fit and evaluate methods to initiate the learning process. Once the results are generated, they are returned to the client proxies and subsequently to the server. Here, the client proxies, and indirectly the Flower server, function as the gRPC client, while the Flower clients act as the gRPC server.

Chapter **5**

Methodology

In this chapter an

5.1 System Architecture

This section will detail the overall architecture of the federated learning system implemented in this study. It will describe the components of the system, how they interact with each other, and the rationale behind architectural choices. The focus will be on illustrating the structure that supports federated learning operations, including client-server communication, data distribution, and model aggregation.

5.1.1 Component Overview

5.1.2 Interaction Flow

5.2 Dataset Preprocessing & Management

In this section the different datasets used and the criteria for their selection. Emphasis will be placed on the preprocessing steps undertaken to prepare the data for analysis, ensuring the data's compatibility with the requirements of federated learning.

5.2.1 Dataset Sources and Selection Criteria

5.2.2 Preprocessing Techniques

5.3 Model Development and Training

for different datasets different model designs are implemented that need to be described. Moreover the custom techniques of the training process has to be explained.

5.3.1 Model Architecture

5.3.2 Training and Optimization

5.4 Federated Learning Implementation

Here, we detail the implementation of the federated learning framework, focusing on the client-server architecture, the distribution of the model across clients, and the aggregation of model

updates. Special attention is given to modifications or custom implementations designed to enhance the efficiency of federated learning in this research, as multiple adjustments needed to be conducted on the initial strategy

5.4.1 Client-Server Setup

5.4.2 Data Partitioning and Distribution

5.4.3 Model Aggregation

5.5 Communication Protocols and Data Serialization

The section discusses the communication protocols established between the clients both the flower and metric server for efficient data exchange and model synchronization. It covers the use of ProtoBuffers to optimize network communication, ensuring data integrity and security during transmission.

5.5.1 Implementation of Communication Protocols

5.5.2 Data Serialization Technique

5.6 Network Simulation Integration

In this section, we delve into the use of the NS3 simulator to replicate real-world network conditions and their impact on federated learning. It details the simulation setup, the parameters adjusted for different network conditions, and how these simulations contribute to understanding federated learning's behavior in varied networking environments.

5.6.1 Simulation Tool Configuration

5.6.2 Simulating Network Conditions

5.6.3 Integration with Federated Learning

5.7 Evaluation Metrics and Performance Analysis

This section outlines the metrics used to evaluate the performance of the federated learning models in conjunction with the network simulation stats.

5.7.1 Model Evaluation Metrics

5.7.2 Network Simulation Metrics

5.7.3 Performance Analysis

Chapter **6**

Experimental Results & Discussion

This chapter presents the findings of the research.

6.1 Overview of Experimental Results

This section provides a high-level summary of the experimental outcomes, setting the stage for the more detailed analyses that follow. It includes initial observations and highlights the key data points that will be explored in depth in subsequent sections.

6.2 Analysis of FL performance

In-depth analysis of the performance of the developed models, focusing on accuracy, efficiency, and scalability. This section discusses the results in the context of the datasets used, the preprocessing techniques applied, and the specific architectures and training optimizations implemented.

6.2.1 Accuracy and Efficiency

6.2.2 Scalability and Robustness

6.3 Network Simulation Findings

Detailed findings from the network simulation using NS3, focusing on how different network conditions impacted the federated learning process. This section evaluates the system's performance in terms of latency, bandwidth usage, and data transmission costs.

6.4 Discussion on Designed System Efficacy

Critical evaluation of the federated learning implementation, discussing its effectiveness in achieving the research objectives. This section contrasts the observed outcomes with the theoretical expectations laid out in earlier chapters, highlighting successes and areas for improvement.

6.4.1 Contributions to Federated Learning Research

6.4.2 Implications for Real-World Applications

Explores the practical implications of the research findings for real-world applications of federated learning. This includes potential use cases in various industries, considerations for deployment, and the impact on data privacy and security.

Industry Use Cases

6.4.3 Comparison with Technologies

6.5 Limitations and Challenges

Reflects on the limitations of the current study and the challenges faced during the research process. This section offers an honest assessment of what could have been done differently and the inherent constraints of the research design.

Part

Epilogue

Chapter **7**

Conclusions

7.1 Research Conclusions

This section summarizes the key findings of the research, emphasizing the major insights gained and how they answer the research questions posed at the beginning of this thesis. It provides a concise overview of the results obtained from the experiments and analyses, discussing the implications of these findings within the context of federated learning, machine learning models, and network simulations.

7.2 Contributions of This Thesis

Summary of Key Findings. In the following section, we highlight the unique contributions made by this thesis to the field of federated learning and machine learning at large.

7.3 Recommendations for Future Research

Limitations of the Study and future work proposals

7.4 Final Thoughts

Bibliography

- [1] Lina Zhou, Shimei Pan, Jianwu Wang και Athanasios V. Vasilakos. *Machine Learning on Big Data: Opportunities and Challenges*. *Neurocomputing*, 237:350–361, 2017.
- [2] Sumit Saha. *A Comprehensive Guide to Convolutional Neural Networks – the ELI5 Way*. <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>, 2022.
- [3] Petrakos Emmanouel. *Reconfigurable Logic (FPGA)-Based System Architecture for the Acceleration of Federated Learning in Neural Networks*. 2023.
- [4] Saad Hikmat Haji και Adnan Mohsin Abdulazeez. *COMPARISON OF OPTIMIZATION TECHNIQUES BASED ON GRADIENT DESCENT ALGORITHM: A REVIEW*. *PalArch's Journal of Archaeology of Egypt / Egyptology*, 18(4):2715–2743, 2021.
- [5] Atulanand. *GRADIENT DESCENT*, 2022.
- [6] Yann LeCun, Leon Bottou, Genevieve B. Orr και Klaus Robert Müller. *Efficient BackProp*. *Neural Networks: Tricks of the Trade* Genevieve B. Orr και Klaus Robert Müller, επιμελητές, σελίδες 9–50. Springer, Berlin, Heidelberg, 1998.
- [7] Loredana Caruccio, Gaetano Cimino, Vincenzo Deufemia, Gianpaolo Iuliano και Roberto Stanzione. *Surveying Federated Learning Approaches through a Multi-Criteria Categorization*. *Multimedia Tools and Applications*, 83(12):36921–36951, 2024.
- [8] Omair Rashed Abdulwareth Almanifi, Chee Onn Chow, Mau Luen Tham, Joon Huang Chuah και Jeevan Kanesan. *Communication and Computation Efficiency in Federated Learning: A Survey*. *Internet of Things*, 22:100742, 2023.
- [9] Izchak Sharfman, Assaf Schuster και Daniel Keren. *A Geometric Approach to Monitoring Threshold Functions over Distributed Data Streams*. *Ubiquitous Knowledge Discovery: Challenges, Techniques, Applications* Michael May και Lorenza Saitta, επιμελητές, σελίδες 163–186. Springer, Berlin, Heidelberg, 2010.
- [10] Siddharth Sharma, Simone Sharma και Anidhya Athaiya. *ACTIVATION FUNCTIONS IN NEURAL NETWORKS*. *International Journal of Engineering Applied Sciences and Technology*, 04(12):310–316, 2020.
- [11] Niklas Kühl, Marc Goutier, Robin Hirt και Gerhard Satzger. *Machine Learning in Artificial Intelligence: Towards a Common Understanding*, 2020.
- [12] *Artificial Intelligence : A Modern Approach*. <https://thuvienso.hoasen.edu.vn/handle/123456789/8967>.
- [13] Haochen Hua, Yutong Li, Tonghe Wang, Nanqing Dong, Wei Li και Junwei Cao. *Edge Computing with Artificial Intelligence: A Machine Learning Perspective*. *ACM Computing Surveys*, 55(9):184:1–184:35, 2023.
- [14] Amit Kumar Tyagi και Rekha G. *Machine Learning with Big Data*, 2019.

- [15] Junfei Qiu, Qihui Wu, Guoru Ding, Yuhua Xu και Shuo Feng. *A Survey of Machine Learning for Big Data Processing*. *EURASIP Journal on Advances in Signal Processing*, 2016(1):67, 2016.
- [16] Iqbal H. Sarker. *Machine Learning: Algorithms, Real-World Applications and Research Directions*. *SN Computer Science*, 2(3):160, 2021.
- [17] Trevor Hastie, Jerome Friedman και Robert Tibshirani. *The Elements of Statistical Learning*, τόμος 9. Springer Science & Business Media, 2017.
- [18] Thrasyvoulos Spyropoulos. *Reinforcement Learning*. *Class at the Department of Electrical and Computer Engineering*, Technical University of Crete, 2023.
- [19] Yoshua Bengio, Aaron Courville και Pascal Vincent. *Representation Learning: A Review and New Perspectives*. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013.
- [20] Vladimir Nasteski. *An Overview of the Supervised Machine Learning Methods*. *HORIZONS.B*, 4:51–62, 2017.
- [21] Zoubin Ghahramani. *Unsupervised Learning*. *Advanced Lectures on Machine Learning: ML Summer Schools 2003, Canberra, Australia, February 2 - 14, 2003, Tübingen, Germany, August 4 - 16, 2003, Revised Lectures* Olivier Bousquet, Ulrike von Luxburg και Gunnar Rätsch, επιμελητές, σελίδες 72–112. Springer, Berlin, Heidelberg, 2004.
- [22] Ekaba Bisong. *Batch vs. Online Learning*. *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners* Ekaba Bisong, επιμελητής, σελίδες 199–201. Apress, Berkeley, CA, 2019.
- [23] Ofer Dekel. *From Online to Batch Learning with Cutoff-Averaging*. *Advances in Neural Information Processing Systems*, τόμος 21. Curran Associates, Inc., 2008.
- [24] Diego Perez. *What Is Model Performance Measurement in Machine Learning?*, 2022.
- [25] Haotian Zhang, Lin Zhang και Yuan Jiang. *Overfitting and Underfitting Analysis for Deep Learning Based End-to-end Communication Systems*. *2019 11th International Conference on Wireless Communications and Signal Processing (WCSP)*, σελίδες 1–6, 2019.
- [26] Diego Perez. *Model Complexity in Machine Learning*, 2022.
- [27] Emily Sullivan. *Understanding from Machine Learning Models*. *The British Journal for the Philosophy of Science*, 73(1):109–133, 2022.
- [28] Machine Learning in Plain English. *Lesson 15 – Machine Learning: Overfitting, Underfitting, and Model Complexity Intuition*, 2023.
- [29] A D Dongare, R R Kharde και Amit D Kachare. *Introduction to Artificial Neural Network*. 2(1), 2012.
- [30] Yu chen Wu και Jun wen Feng. *Development and Application of Artificial Neural Network*. *Wireless Personal Communications*, 102(2):1645–1656, 2018.
- [31] F. Rosenblatt. *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*. *Psychological Review*, 65(6):386–408, 1958.
- [32] Zhihua Zhang. *Artificial Neural Network*. *Multivariate Time Series Analysis in Climate and Environmental Research* Zhihua Zhang, επιμελητής, σελίδες 1–35. Springer International Publishing, Cham, 2018.

-
- [33] Marius Constantin Popescu, Valentina E Balas, Liliana Perescu-Popescu και Nikos Mastorakis. *Multilayer Perceptron and Neural Networks*. 8(7), 2009.
 - [34] Marvin Minsky και Seymour A. Papert. *Perceptrons, Reissue of the 1988 Expanded Edition with a New Foreword by Léon Bottou: An Introduction to Computational Geometry*. MIT Press, 2017.
 - [35] *The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions* | *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*. <https://www.worldscientific.com/doi/abs/10.1142/s0218488598000094>.
 - [36] Shiv Ram Dubey, Satish Kumar Singh και Bidyut Baran Chaudhuri. *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. *Neurocomputing*, 503:92–108, 2022.
 - [37] Yasara Ransisi. *Neural Network Types.*, 2021.
 - [38] Keiron O’Shea και Ryan Nash. *An Introduction to Convolutional Neural Networks*, 2015.
 - [39] ODSC Community. *Understanding the Mechanism and Types of Recurrent Neural Networks*, 2020.
 - [40] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus και Shimon Schocken. *Multilayer Feedforward Networks with a Nonpolynomial Activation Function Can Approximate Any Function*. *Neural Networks*, 6(6):861–867, 1993.
 - [41] M. Ghiassi, H. Saidane και D. K. Zimbra. *A Dynamic Artificial Neural Network Model for Forecasting Time Series Events*. *International Journal of Forecasting*, 21(2):341–362, 2005.
 - [42] Jiuxiang Gu, Zhenhua Wang, Jason Kuen, Lianyang Ma, Amir Shahroudy, Bing Shuai, Ting Liu, Xingxing Wang, Gang Wang, Jianfei Cai και Tsuhan Chen. *Recent Advances in Convolutional Neural Networks*. *Pattern Recognition*, 77:354–377, 2018.
 - [43] Cyber Marty. *Neural Networks Learning Process Explained*, 2023.
 - [44] Ian Goodfellow, Yoshua Bengio και Aaron Courville. *Deep Learning*. MIT Press, 2016.
 - [45] Meenal V. Narkhede, Prashant P. Bartakke και Mukul S. Sutaone. *A Review on Weight Initialization Strategies for Neural Networks*. *Artificial Intelligence Review*, 55(1):291–322, 2022.
 - [46] Michael Nielsen. *Neural Networks and Deep Learning*.
 - [47] Mirza Cilimkovic. *Neural Networks and Back Propagation Algorithm*.
 - [48] Sebastian Ruder. *An Overview of Gradient Descent Optimization Algorithms*, 2017.
 - [49] *Momentum and Learning Rate Adaptation*. <https://willamette.edu/~gorr/classes/cs449/momrate.html>.
 - [50] Diederik P. Kingma και Jimmy Ba. *Adam: A Method for Stochastic Optimization*, 2017.
 - [51] John Duchi, Elad Hazan και Yoram Singer. *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*.
 - [52] *Lecture 6.5-rmsprop: Divide the Gradient by a Running Average of Its Recent Magnitude* | *CiNii Research*. <https://cir.nii.ac.jp/crid/1370017282431050757>.

- [53] Joost Verbraeken, Matthijs Wolting, Jonathan Katzy, Jeroen Kloppenburg και Tim Verbelen. *A Survey on Distributed Machine Learning* | *ACM Computing Surveys*. <https://dl.acm.org/doi/abs/10.1145/3377454>.
- [54] Ling Liu, Pan Zhou, Gang Sun, Xi Chen, Tao Wu, Hongfang Yu και Mohsen Guizani. *Topologies in Distributed Machine Learning: Comprehensive Survey, Recommendations and Future Directions*. *Neurocomputing*, 567:127009, 2024.
- [55] Alexander Sergeev και Mike Del Balso. *Horovod: Fast and Easy Distributed Deep Learning in TensorFlow*, 2018.
- [56] *O'Reilly Network: Distributed Systems Topologies: Part 2*.
- [57] Surat Teerapittayanon, Bradley McDanel και H.T. Kung. *Distributed Deep Neural Networks Over the Cloud, the Edge and End Devices*. *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, σελίδες 328–339, Atlanta, GA, USA, 2017. IEEE.
- [58] Jianmin Chen, Xinghao Pan, Rajat Monga, Samy Bengio και Rafal Jozefowicz. *Revisiting Distributed Synchronous SGD*, 2017.
- [59] *Asynchronous Decentralized Parallel Stochastic Gradient Descent*. <https://proceedings.mlr.press/v80/lian18a.html>.
- [60] Zheng Xu, Yanxiang Zhang, Galen Andrew, Christopher A. Choquette-Choo, Peter Kairouz, H. Brendan McMahan, Jesse Rosenstock και Yuanbo Zhang. *Federated Learning of Gboard Language Models with Differential Privacy*, 2023.
- [61] Cloud Hacks. *Federated Learning: A Paradigm Shift in Data Privacy and Model Training*, 2024.
- [62] *The General Data Protection Regulation*. <https://www.consilium.europa.eu/en/policies/data-protection/data-protection-regulation/>.
- [63] *California Consumer Privacy Act (CCPA)* | *State of California - Department of Justice - Office of the Attorney General*. <https://oag.ca.gov/privacy/ccpa>.
- [64] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson και Blaise Agüeray Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*, 2023.
- [65] Mingzhe Chen, Deniz Gündüz, Kaibin Huang, Walid Saad, Mehdi Bennis, Aneta Vulgarakis Feljan και H. Vincent Poor. *Distributed Learning in Wireless Networks: Recent Progress and Future Challenges*. *IEEE Journal on Selected Areas in Communications*, 39(12):3579–3605, 2021.
- [66] Qiang Yang, Yang Liu, Tianjian Chen και Yongxin Tong. *Federated Machine Learning: Concept and Applications*, 2019.
- [67] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin και Vikas Chandra. *Federated Learning with Non-IID Data*. 2018.
- [68] Jakub Konečný, H. Brendan McMahan, Daniel Ramage και Peter Richtárik. *Federated Optimization: Distributed Machine Learning for On-Device Intelligence*, 2016.
- [69] Rajarshi Bhose. *Differentiating between Distributed Learning and Federated Learning*, 2023.

- [70] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc Le και Andrew Ng. *Large Scale Distributed Deep Networks*. *Advances in Neural Information Processing Systems*, τόμος 25. Curran Associates, Inc., 2012.
- [71] Myeongsuk Pak και Sanghoon Kim. *A Review of Deep Learning in Image Recognition*.
- [72] Y. Lecun, L. Bottou, Y. Bengio και P. Haffner. *Gradient-Based Learning Applied to Document Recognition*. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [73] Karen Simonyan και Andrew Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*, 2015.
- [74] Kaiming He, Xiangyu Zhang, Shaoqing Ren και Jian Sun. *Deep Residual Learning for Image Recognition*, 2015.
- [75] Tian Li, Anit Kumar Sahu, Ameet Talwalkar και Virginia Smith. *Federated Learning: Challenges, Methods, and Future Directions*. *IEEE Signal Processing Magazine*, 37(3):50–60, 2020.
- [76] Minos Garofalakis, Daniel Keren και Vasilis Samoladas. *Sketch-Based Geometric Monitoring of Distributed Stream Queries*. *Proceedings of the VLDB Endowment*, 6(10):937–948, 2013.
- [77] Vasileios Samoladas και Minos Garofalakis. *Functional Geometric Monitoring for Distributed Streams*. OpenProceedings.org, 2019.
- [78] Alon Noga, Matias Yossi και Szegedy Mario. *The Space Complexity of Approximating the Frequency Moments*. 1996.
- [79] Konidaris Vissarion. *Extreme-Scale Online Machine Learning OnStream Processing Platforms*. 2022.
- [80] Theologitis Michael. *Algorithms for Online Federated Machine Learning*. 2023.
- [81] Frangias Georgios. *Federated Learning at TensorFlow Using the Geometric Approach*. 2023.
- [82] Emily Ekaireb, Xiaofan Yu, Kazim Ergun, Quanling Zhao, Kai Lee, Muhammad Huzaira και Tajana Rosing. *Ns3-FL: Simulating Federated Learning with Ns-3*. *Proceedings of the 2022 Workshop on Ns-3*, σελίδες 97–104, Virtual Event USA, 2022. ACM.